

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_cffc9fdf-8f2\agent-aceea7c9bb3c38c9c.jsonl`  
- SHA-256: `f782b44d10ea83efb21e45c90474335b7195f169fc0d38c21261e9849a8d2378`  
- Source modified: `2026-06-12T09:05:51+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_cffc9fdf-8f2`  
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T09:03:26.893Z)

The three sibling repos (all are accessible working directories):

- C:\Users\avidu\Projects\badminton-highlight-indexer (main indexer: FastAPI + ffmpeg pipeline)
- C:\Users\avidu\Projects\sports-data-collector (data collection; believed to hold the Roora vendor ingestion flow)
- C:\Users\avidu\Projects\rally-annotator (VLC-based annotation tool)

TASK: Map the end-to-end Roora vendor handoff flow: how videos are selected/prepared/delivered to the vendor for annotation, and how their annotations come back and get ingested.

Find (likely mostly in sports-data-collector, plus docs in badminton-highlight-indexer):

1. The annotation guideline doc given to Roora (what tool do THEY use? what video format/resolution do they receive today? any spec we gave them?). Any mention of what resolution/fps their tools are comfortable with.
2. The ingestion path for vendor annotations (parser, schema, timestamp units ? seconds? frames? hh:mm:ss?), and how annotations are KEYED back to a video (filename? video_id/MD5? something else?). This is critical: if we hand Roora a re-encoded copy, do returned annotations still join correctly against the ORIGINAL video?
3. Any timestamp normalization/offset handling already in the ingestion code (e.g. configurable per-batch offset correction).
4. The GS-1..GS-4 golden-set batches: what was actually sent (file format/resolution), and any retro notes about quality issues with what came back.

Report concrete file paths + line ranges, the exact join key for annotations->video, timestamp unit/precision, and anything documented about the vendor's tooling constraints.

2. assistant (2026-06-12T09:03:28.942Z)

I'll help you map the end-to-end Roora vendor handoff flow. Let me start by exploring the three repositories to find the relevant code and documentation.

3. user (2026-06-12T09:03:30.876Z)

```
020ff61 docs: status sync - PR-2 MERGED (#99, live-E2E-verified), sweep recorded, BACKLOG shlex  
item closed (#100)  
a5982e5 docs: status sync - PR-2 MERGED (#99, live-E2E-verified), sweep recorded, BACKLOG shlex  
item closed  
5eaba74 feat(api): chunked upload + highlight download (B2+B3) - static UI compile fix +  
download link (#99)  
e2ddc65 feat(api): chunked upload + highlight download (B2+B3) - static UI compile fix +  
download link  
31069cb fix(detectors): keep ~ expandable in WSL staging/inference commands (#97)  
7717e15 ci: bump checkout and setup-python to Node-24 majors (v6) (#98)  
3d84869 ci: bump checkout and setup-python to Node-24 majors (v6)  
281a81d fix(detectors): abspath the output dir before /mnt translation
```

70ca6fa fix(detectors): keep ~ expandable in WSL staging/inference commands
2c241a8 refactor: collapse the hybrid-segmenter twins into a shared trajectory base
99b84c9 feat(config): warn loudly on unknown config.json keys (dotted paths) (#96)
f4cdfa5 feat(config): warn loudly on unknown config.json keys (dotted paths)
7d007af ci: coverage floor on the full-suite lane (ratchet-only) (#95)
26b4fb7 ci: pin the coverage floor at 70 (CI-measured 72%)
355192e ci: run the full suite with coverage (report only; floor follows)
1053505 fix(db): actually close SQLite connections (transaction scope != close) (#94)
c1c940d fix(db): actually close SQLite connections (transaction scope != close)
df9cf80 ci: add mypy lane (lenient green baseline + explicit quarantine list) (#93)
f49ce9b ci: add mypy lane (lenient green baseline + explicit quarantine list)
595a5a1 ci: add ruff lint lane + fix all violations; remove never-compiled flyer stub (#92)

4. assistant (2026-06-12T09:03:33.116Z)

Let me explore the sports-data-collector repository for Roora vendor-related code and documentation:

5. user (2026-06-12T09:03:33.625Z)

```
C:\Users\avidu\Projects\sports-data-collector/.pytest_cache/README.md
C:\Users\avidu\Projects\sports-data-collector/cleanup.py
C:\Users\avidu\Projects\sports-data-collector/data/eval_export/manifest.json
C:\Users\avidu\Projects\sports-data-collector/data/mock_local_badminton_anno.json
C:\Users\avidu\Projects\sports-data-collector/data/roora_badminton_issues.txt
C:\Users\avidu\Projects\sports-data-collector/data/validation/pilot_badminton_01_report.json
C:\Users\avidu\Projects\sports-data-collector/data/validation/pilot_badminton_01_report.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/NDA_TEMPLATE.md
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ROORA_BRIEF.md
C:\Users\avidu\Projects\sports-data-collector/docs/DECISIONS.md
C:\Users\avidu\Projects\sports-data-collector/docs/INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md
C:\Users\avidu\Projects\sports-data-collector/docs/PIPELINE_ROLE.md
C:\Users\avidu\Projects\sports-data-collector/README.md
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py
C:\Users\avidu\Projects\sports-data-collector/src/cli/ingest.py
C:\Users\avidu\Projects\sports-data-collector/src/cli/normalize_annotations.py
C:\Users\avidu\Projects\sports-data-collector/src/core/config.py
C:\Users\avidu\Projects\sports-data-collector/src/core/db.py
C:\Users\avidu\Projects\sports-data-collector/src/core/downloader.py
C:\Users\avidu\Projects\sports-data-collector/src/core/parsing.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/badminton.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/base.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/cricket.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/pickleball.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/tabletennis.py
C:\Users\avidu\Projects\sports-data-collector/src/core/schemas/tennis.py
C:\Users\avidu\Projects\sports-data-collector/src/core/validator.py
C:\Users\avidu\Projects\sports-data-collector/src/crawlers/badminton_crawler.py
```

6. assistant (2026-06-12T09:03:35.451Z)

Excellent! I found several key files. Let me read them in parallel:

7. user (2026-06-12T09:03:35.690Z)

```
1      # Roora Annotation Brief ? Rally Golden Set
2
3      **Project:** racket-sport rally segmentation golden / regression dataset
4      **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5      **Companion doc:** [ `ANNOTATION_GUIDE.md` ] (./ANNOTATION_GUIDE.md)
6
```

```

7      > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8      > imports directly via `python -m src.cli.curate import-local`.
9
10     ## 0. What we need in one sentence
11     For each provided match video, produce one CSV file listing every rally with its
12     start time, end time, number of shots, and how it ended ? excluding all dead time.
13
14     ## 1. Deliverable format (canonical)
15     One CSV per video (UTF-8), named exactly `.csv` (we supply the `video_id`).
16     One row per rally, chronological, header row required.
17
18     | column | meaning |
19     |---|---|
20     | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21     | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22     | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23     | `start_time` | decimal seconds (e.g. `72.40`). Auto-filled from `start_mmss`.
|
24     | `end_time` | decimal seconds. Auto-filled from `end_mmss`. |
25     | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26     | `ending_reason` | one of: `winner` \| `forced_error` \| `unforced_error` \|
`service_fault` \| `let` \| `other` (why it ended) |
27     | `last_shot_outcome` | one of: `in` \| `net` \| `out` \| `n_a` (where the last shot
landed; `n_a` for service_fault/let) |
28     | `score_before` | optional, "A-B". Blank if not confidently readable. |
29     | `score_after` | optional, "A-B". |
30     | `notes` | optional free text. Required when `ending_reason = other`. |
31
32     Times (mm:ss ? decimal seconds). Raters type only `start_mmss`/`end_mmss`. The
template
33     Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
row 2):
34
35     ```
36     =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
37     ```
38
39     On Download as CSV, the computed decimal seconds are exported. (If your tool already
40     shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
columns blank.)
41
42     Example (badminton):
43     ```csv
44     rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
_outcome,score_before,score_after,notes
45     1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
46     2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
47     3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
pressure)
48     ```
49
50     Critical format rules
51     - `start_time`/`end_time` that reach us must be decimal seconds (the formula
guarantees this).
52     - Times are relative to the start of the file we provide. Do not trim, clip, or
53     re-encode the video ? our pipeline identifies a video by its file checksum.
54     - `end_time` > `start_time` for every row.
55     - No overlaps: `end_time[N]` <= `start_time[N+1]`.
56     - Everything between one rally's `end_time` and the next rally's `start_time` is dead
time and gets no row.
57
58     ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
59     1. Ignore all dead time. Only label active play.

```

```

60      2. **Include every rally ? exclude none.** Every served point, including **service
faults**,
61      1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
project.
62      **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
up looks
63      like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
serve,
64      cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
65      in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
66      3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
67
68      ## 3. Scope & phasing
69      **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
tennis),
70      each `< 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review
jointly and
71      lock the Guide before scale-up.
72
73      **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
total), each
74      ~30 min. Same format, rules, and template.
75
76      > **Doubles are included** across the program. Rally rules are identical to singles; the
77      > badminton- and pickleball-specific doubles notes are in the Guide.
78
79      ## 4. Quality bar (acceptance criteria)
80      Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81      ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82      - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83      - **Boundaries:** `start_time`/`end_time` within **±0.3 s** of true serve-contact /
landing.
84      - **shots_count:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
85      - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86      - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88      ## 5. Per-sport quick reference (full version + examples in the Guide)
89      **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle
floors / fault · shot: each racket?shuttle contact.
90      **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double
bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are **not** shots;
two-bounce rule is normal play).
91      **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal
return · shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
`let`.
92
93      ## 6. ending_reason values (decision tree in the Guide)
94      This vocabulary attributes **why** the point ended; it is shared across all
95      net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
96      the last shot physically landed in the separate **`last_shot_outcome`** column
97      (**`in` \| **`net` \| **`out` \| **`n_a`) ? e.g. a forced error that flew long is
98      **`ending_reason=forced_error, last_shot_outcome=out`.
99
100     | value | meaning |
101     |---|---|
102     | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
winner). |
103     | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104     | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
time and position to make. |

```

```

105 | `service_fault` | the serve itself failed / was illegal. |
106 | `let` | the point was replayed. |
107 | `other` | anything else / cannot tell. **Must** add a note. |
108
109 ## 7. Recommended capture method
110 Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111 **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112 (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113 `seconds = frame_number / fps` (we provide fps per video).
114
115 ## 8. Logistics & contacts
116 - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117 - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118 - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
119 - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120 - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121 - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122 - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124 ## 9. Pricing request (please quote)
125 1. Price **per rally** and **per minute** of footage.
126 2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127 3. Indicative **scale** price for ~36?45 longer videos.
128 4. Turnaround for the pilot and throughput (rallies/day) at scale.
129 5. QA process and rework policy.
130 6. Whether annotators have racket-sport familiarity (helpful, not required).
131 7. Confirmation you can deliver the CSV schema above.
132

```

8. user (2026-06-12T09:03:35.920Z)

```

1 # Illustrated Annotation Guide ? Rally Labeling
2
3 Supplement to [`ROORA_BRIEF.md`](./ROORA_BRIEF.md). Sports: **Badminton, Pickleball,
Table Tennis** (singles **and** doubles).
4
5 > **Illustrations:** diagrams are drawn in text so they survive copy/paste. Boxes marked
6 > [`[SCREENSHOT: ...]` are placeholders.
7 > **TODO(avidullu): replace every [`[SCREENSHOT: ...]` placeholder with a real frame grab
8 > from the pilot videos before sending this guide to Roora.**
9
10 ## Part 1 ? What you are producing
11 One CSV per video (from the template Sheet), one row per rally:
12
13 ```
14 rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
last_shot_outcome, score_before, score_after, notes
15 ```
16
17 You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason`,
18 `last_shot_outcome` (+ optional score/notes). The `start_time`/`end_time` decimal-second
columns
19 **fill themselves** via a formula already in the template.
20
21 For every rally you record:
22 - **When it started** (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)

```

```

23 - **When it ended** (shuttle/ball dead) ? `end_mmss`
24 - **How many shots** happened ? `shots_count`
25 - **Why it ended** ? `ending_reason` . **Where the last shot landed** ?
`last_shot_outcome`
26
27 Everything else (gaps between rallies) you **ignore**.
28
29 ## Part 2 ? The three golden rules
30
31 ### Rule 1: Ignore all dead time
32 A rally is **continuous active play**. The moment play stops, the rally is over. The
time
33 until the next serve is **dead time** and is never labeled.
34
35 ...
36 DEAD RALLY 1 DEAD RALLY 2 DEAD
37 (ignore) |=====| (ignore) |=====| (ignore)
38 ^ ^ ^ ^
39 serve shuttle serve shuttle
40 contact lands contact lands
41 =start_1 =end_1 =start_2 =end_2
42 ...
43
44 Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat,
45 bouncing the ball before serving, the score being announced, replays/slow-mo, crowd or
coach
46 shots, changeovers, towel/medical timeouts, and **pre-match / between-games warm-up
47 (knock-up)** ? see the Rule 2 caveat below.
48
49 ### Rule 2: Include every rally ? even the ugly ones
50 Every served point gets a row, including:
51 - **Service faults** (serve into net/out/illegal) ? usually `shots_count = 1`,
`ending_reason = service_fault`.
52 - **Very short rallies** (serve + one error).
53 - **Lets / replays** ? `ending_reason = let`; the replayed point is its **own** next
row.
54
55 A **missed rally is the worst error** in this project ? the model must learn that short
and
56 failed rallies are still rallies.
57
58 > **?? BUT warm-up / knock-up is NOT a rally** ? a common trap, because Rule 2 says
"include
59 > everything." Pre-match and between-games warm-up *looks* like rallying but is not
scored play;
60 > **do not label it.** How to tell it apart:
61 > - **No real serve.** Real play starts with a proper serve from the service position
(diagonal,
62 > behind the service line). Warm-up has players feeding cooperatively from mid-court
with no
63 > serve ? not a rally.
64 > - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is
trying to win
65 > the point (no smashes-to-win, no scrambling/diving).
66 > - **Score isn't progressing.** No umpire / scoreboard not running / players casually
retrieve
67 > and re-feed; the score doesn't change. Often **several shuttles/balls** in use at
once.
68 > - **Per sport:** badminton & pickleball ? real play starts with the underhand serve
from the
69 > service court / behind the baseline; table tennis ? real play starts with the
tossed,
70 > two-bounce serve (knock-up is cooperative rallying without it).
71 >
72 > **When unsure:** find the first proper serve where the score starts to progress ?

```

```

labeling
73     > begins there. If you genuinely can't tell whether the match has started, **flag the
clip to us
74     > ? don't guess.**
75
76     ### Rule 3: A shot = one contact with the racket / paddle / bat
77     Count every time the shuttle/ball touches a racket, paddle, or bat. The serve is shot 1.
78     - **Count:** any racket/paddle/bat contact (serve, return, smash, block?).
79     - **Don't count:** floor bounces, table bounces, net clips, the toss before a serve.
80
81     5-shot example: `serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5)` ? `shots_count =
5`, `ending_reason = winner`.
82
83     ## Part 3 ? Where exactly does a rally start and end?
84     **START (`start_mmss`):** the exact frame the server's racket/paddle/bat **contacts**
the
85     shuttle/ball. Not the toss, not the wind-up ? the contact frame.
86     `[SCREENSHOT: serve-contact frame]`
87
88     **END (`end_mmss`):** the exact frame the shuttle/ball becomes **dead**:
89     - Badminton: shuttle touches the floor, or play stops on a fault.
90     - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
91     - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.
92
93     `[SCREENSHOT: end frame]`
94
95     If occluded/off-screen, pick the closest clear frame and add a note.
96     **Boundary tolerance:  $\pm 0.3$  s** ( $\sim \pm 9$  frames at 30 fps).
97
98     ## Part 4 ? ending_reason: how to choose
99     Pick exactly **one** value for how the rally ended. Stop at the first match:
100
101     1. Did the **serve** itself fail / was illegal (and ended the point)? ? `service_fault`
102     2. Was the point **replayed** (let / interruption)? ? `let`
103     3. Did the final shot land **in** and the opponent make **no legal contact** ? ? `winner`
(a clean winner)
104     4. Did the losing player make an error (netted / out / missed) **under pressure** ?
105     stretched, wrong-footed, or rushed by a strong shot? ? `forced_error`
106     5. Did the losing player make an error on a **routine, unpressured** ball they had
107     the time and position to make? ? `unforced_error`
108     6. None of the above / cannot tell ? `other` (add a note)
109
110     **Forced vs unforced ? the one judgment call.** Decide using the opponent's *preceding*
111     shot:
112     - `forced_error` = the opponent's shot **caused** the miss (pace, placement, deception;
113     the player barely reached it or was pulled out of position).
114     - `unforced_error` = a clean, makeable ball the player missed on their own.
115     - When genuinely unsure between the two, choose `unforced_error` and add a brief note.
116
117     Notes:
118     - A clean `winner` means **no legal contact** by the opponent. If they got a racket /
paddle /
119     bat on it but netted or hit out, that is a `forced_error` / `unforced_error` (their
shot), not a `winner`.
120     - A serve into the net is `service_fault` (rule 1 beats everything).
121
122     ### Then set `last_shot_outcome` ? where the last shot physically went
123     This is a **separate, observational** column (no judgment): just look at the last shot.
124     - `in` ? landed in (winners; also a put-away the opponent couldn't reach).
125     - `net` ? went into the net.
126     - `out` ? landed outside the court / past the lines.
127     - `n_a` ? not applicable: use for `service_fault` and `let`.
128
129     So each rally gets both: **why** (`ending_reason`) and **where** (`last_shot_outcome`) ?
e.g. a

```

```

130     smash winner = `winner, in`; a clear forced long = `forced_error, out`; a routine ball
netted =
131     `unforced_error, net`.
132
133     ## Part 5 ? Per-sport detail
134
135     ### Badminton
136     - **Serve:** underhand, below the waist, diagonal. Start = racket?shuttle contact.
137     - **Rally end:** shuttle hits the floor, or play stops on a fault.
138     - **Shots:** every racket?shuttle contact; serve = shot 1.
139     - **Service faults** (1-shot rally, `service_fault`): serve into net, serve outside
service court, contact above waist, foot fault.
140
141     Worked example:
142     ```
143     [SCREENSHOT: badminton serve]
144     Rally 4: serve 1:28.30, long rally, smash winner, shuttle lands 1:44.10, 12 contacts.
145           4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, 3-5, 4-5,
146     Rally 5: serve 1:57.00 straight into the net.
147           5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, 4-5, 4-6,
148     ```
149
150     **Badminton ? DOUBLES (2 players per side).** All rally rules are **identical** to
singles:
151     - `start` = serve contact; `end` = shuttle down / fault; a **shot = any racket?shuttle
contact by either player on a side**.
152     - `shots_count` = **total** contacts across both teams in the rally. **Do not track
which player hit** ? just count every contact.
153     - Net exchanges (drives, flat pushes) come fast ? **frame-step** so you neither miss nor
double-count contacts.
154     - Only the **designated receiver** returns the serve; if the wrong player returns, the
rally ends on a fault ? still a row (`service_fault` if it's a serve/receiving-position fault,
else `other` + note).
155     - Two teammates may **not** both hit the shuttle in succession (double-hit). If you see
two contacts on the **same** side back-to-back, that's a fault ending the rally ? count both
contacts, then the rally ends.
156
157     ### Pickleball
158     - **Serve:** underhand, paddle below waist, hit diagonally; must clear the non-volley
zone (the "kitchen"). Start = paddle?ball contact.
159     - **Two-bounce rule:** the serve must bounce once in the receiver's court and the return
must bounce too, before either side may volley. These bounces are **normal play**, not rally
ends, and bounces are **not** shots.
160     - **Rally end:** ball bounces twice, lands out, hits the net (fails to cross), or a
fault (e.g. volleying in the kitchen).
161     - **Shots:** every paddle?ball contact only; serve = shot 1; floor bounces are not
shots.
162     - **Service faults** (`service_fault`): serve into net, out/long, short in the kitchen,
foot fault.
163
164     Worked example:
165     ```
166     [SCREENSHOT: pickleball serve + kitchen line]
167     Rally 2: serve 0:33.10; serve bounces, return bounces, a few volleys; receiver nets an
easy one at 0:41.85; 6 paddle contacts.
168           2, 0:33.10, 0:41.85, (auto), (auto), 6, unforced_error, 1-0, 2-0, easy ball netted
169     ```
170
171     **Pickleball ? DOUBLES (the standard format, 2 per side).** All rally rules
**identical** to singles:
172     - `start` = serve contact; a **shot = any paddle?ball contact by either teammate**;
floor bounces still not shots; two-bounce rule still applies.
173     - `shots_count` = **total** paddle contacts across both teams; don't track which player.
174     - **Serving sequence (FYI only ? you do NOT annotate it):** both partners serve before
the serve passes to the opponents, except the very first service turn of the game. The called

```


score has three numbers (server score ? receiver score ? server #1/#2). Ignore this for annotation; just mark boundaries / shots / ending.

175 - If both teammates contact the ball in succession, that's a fault ? rally ends; count the contacts, set the ending per the fault.

176

177 **### Table tennis**

178 - ****Serve:**** ball tossed up from an open palm, then struck so it bounces once on each side. Start = the ****racket?ball strike**** (not the toss).

179 - ****Rally end:**** a player fails a legal return ? ball double-bounces on their side, goes off the table, or is hit into the net on a non-serve shot.

180 - ****Shots:**** every racket?ball contact only; serve = shot 1; table bounces are not shots.

181 - ****Let:**** a serve that clips the net but is otherwise legal ? the point is replayed. `ending\_reason = let`, and the replay is its own next row.

182 - ****Service faults**** (`service\_fault`): missed/illegal toss, serve into net, serve off the table.

183 - ****Speed warning:**** contacts can be < 0.2 s apart ? ****step frame-by-frame**** to count shots.

184 - ****Doubles (if a clip appears):**** partners must hit in ****strict alternation**** and serve diagonally; a missed alternation is a fault that ends the rally. Same rule ? count ****every**** racket?ball contact as a shot; `shots\_count` is the team total.

185

186 **Worked example:**

187 ```

188 [SCREENSHOT: table-tennis serve contact]

189 Rally 9: serve 4:10.40; fast 7-contact rally; forehand winner, opponent no contact; dead 4:14.05.

190 9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, 6-9, 6-10,

191 Rally 10: serve 4:22.10 clips the net (otherwise good) -> let.

192 10, 4:22.10, 4:23.00, (auto), (auto), 1, let, 6-10, 6-10, net-cord serve, replayed

193 ```

194

195 **## Part 6 ? Edge cases & FAQ**

196 - ****Two rallies with almost no gap:**** Still two rows. `end\_time[N] <= start\_time[N+1]`, never overlap.

197 - ****Interrupted & replayed:**** Label the interrupted attempt as its own row (`ending\_reason=let`), then the replayed point as the next row.

198 - ****Serve tossed but caught / re-served (no contact):**** No contact = no rally yet. Start at the actual serve contact; don't label the aborted toss.

199 - ****Opponent reached the ball but netted/hit out:**** That's a `forced\_error`/`unforced\_error` (their shot), not `winner` ? `forced` if your shot pressured them, `unforced` if it was routine.

200 - ****Can't read the scoreboard:**** Leave `score\_before`/`score\_after` blank. Never guess.

201 - ****"Let" where the rules say play on:**** If play continued, it's ****not**** a let ? keep one rally and judge the real ending.

202 - ****Warm-up / knock-up / practice points:**** Not real rallies ? ****do not label them.**** Start at the first real, scored serve. See the ****Rule 2 caveat (Part 2)**** for how to tell warm-up from real play (no serve, cooperative not competitive, score not progressing, several shuttles/balls in use). When unsure whether the match has started, flag the clip ? don't guess.

203 - ****Doubles:**** Same rules ? a shot is any one racket/paddle contact by whichever player hits it. See the badminton and pickleball ****DOUBLES**** notes in Part 5; never attribute `shots\_count` to an individual, just total the contacts.

204 - ****Server completely whiffs (no contact):**** Rare. True zero contact ? `shots\_count = 0`, `ending\_reason = service\_fault`, add a note. A graze = 1 contact = 1 shot.

205

206 **## Part 7 ? Self-check before you submit (per video)**

207 - [] `rally\_number` is 1,2,3? no gaps or repeats.

208 - [] Every `start\_time < end\_time` (check the auto-filled decimal columns populated).

209 - [] No overlaps: `end\_time[N] <= start\_time[N+1]`.

210 - [] Times entered as `m:ss.mmm`; decimal `start\_time`/`end\_time` auto-filled and look right.

211 - [] No dead-time rows (scan the gaps ? dead time, not missing rallies).

212 - [] Every served point present, including service faults and lets.

213 - [] `shots\_count` looks right (re-step any rally you rushed; doubles = team total).

214 - [] `ending\_reason` ?

```
{`winner`,`forced_error`,`unforced_error`,`service_fault`,`let`,`other`}; notes filled when
`other`.
215 - [ ] `last_shot_outcome` ? {`in`,`net`,`out`,`n_a`} (`n_a` for service_fault/let).
216 - [ ] File downloaded as `.csv`, header row present, UTF-8.
217 - [ ] Did **not** trim / re-encode / rename the source video.
218
219 ## Part 8 ? Mini glossary
220 - **Rally** ? one continuous passage of play from serve contact to the shuttle/ball
dead.
221 - **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either
teammate in doubles).
222 - **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
223 - **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
224 - **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
225 - **Winner / forced_error / unforced_error** ? the final shot was a clean winner
(opponent made no legal contact) / the loser's error was forced by a pressuring shot / the
loser's error was on a routine, unpressured ball.
226 - **start_time / end_time** ? decimal seconds from the start of the provided file (auto-
filled from mm:ss).
227
```

9. user (2026-06-12T09:03:36.119Z)

```
1 # Rally Ingestion Pipeline ? End-to-End FAQ
2
3 How a vendor's rally annotations become training/eval ground truth, and how we
4 validate each delivery against our conventions. This is the operational companion
5 to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).
6
7 ...
8 vendor delivery OUR side (collector) modeling
(indexer)
9 ?????????????????? convert ?????????????????? validate ??????????????????
10 ? CVAT export ? ?????????????? ? canonical CSV ? ?????????? ? report .md ?
11 ? (boxes + attrs)? ? (per rally, ? ? report .json ?
12 ? OR ? ? decimal secs)? ? review .csv ??? human
verifies
13 ? canonical CSV ? ?????????????????? ??????????????????
flagged rallies
14 ?????????????????? ? ?
15 ? import-local (validated) ?
16 ? golden labels
17 ?????????????????? export ??????????????????
18 ? SQLite (rally ? ?????????? ? <id>.csv ?
start,end (indexer GT)
19 ? tables) ? ? manifest.json?
20 ?????????????????? ??????????????????
21 ?
22 slice clips / calibrate /
train
23 ...
24
25 ## The four commands (in order)
26
27 ```bash
28 # 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a
CSV)
29 python -m src.cli.curate convert-cvat --input job.xml \
30 --video-path match.mp4 --out canonical.csv --issues issues.txt
31
32 # 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
33 python -m src.cli.curate validate-delivery --input job.xml \
34 --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
35 # ...or validate a CSV directly:
36 python -m src.cli.curate validate-delivery --csv canonical.csv --video-id
```

```

pilot_badminton_01
37
38 # 3. After human review, import the (corrected) CSV into the DB
39 python -m src.cli.curate import-local --sport badminton \
40     --video-path match.mp4 --annotations-path canonical.csv --fps 59.94
41
42 # 4. Export indexer-ready ground truth (start,end CSVs + manifest)
43 python -m src.cli.curate export --out-dir data/eval_export
44 ```
45
46 ---
47
48 ## FAQ
49
50 ### What delivery formats do we accept?
51 Both. A CVAT export (image- or track-mode XML ? rally fields carried as box
52 attributes) is normalised by `convert-cvat`; an already-canonical CSV is taken
53 as-is. Vendors may keep working in CVAT ? we do not require them to hand-author
54 CSVs.
55
56 ### What is the "canonical" CSV?
57 One row per rally:
58 `rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
59 last_shot_outcome, score_before, score_after, notes`.
60 `start_time`/`end_time` are decimal seconds; `import-local` reads those.
61
62 ### How are timestamps handled? (the fps thing)
63 We recompute each rally's interval from its frame range × the true fps
64 (`seconds = frame / fps`). A vendor's CVAT time attributes are ignored ? a
65 step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
66 from `--fps` or fprobe of `--video-path`. `mm:ss` is also accepted everywhere.
67
68 ### What does validation check?
69 | code | severity | meaning |
70 |---|---|---|
71 | `overlap` | blocker | two rallies' intervals overlap (`end[N] > start[N+1]`) |
72 | `zero_duration` | blocker | `end <= start` (single-sample / stray box) |
73 | `frame_out_of_range` | high | a box frame is past the video's last frame |
74 | `shot_density` | medium | long rally with implausibly few shots ? likely an inflated
75 span (often hides a merged rally) |
76 | `possible_missed_rally` | medium | a long unlabeled gap between two rallies ? check
77 for an omitted rally |
78 | `ending_reason_vocab` | medium | `ending_reason` outside the agreed set |
79 | `outcome_vocab` | low | `last_shot_outcome` outside `{in,net,out,n_a}` |
80
81 ### Blocker vs. review ? what's the gate?
82 `validate-delivery` exits non-zero if any blocker exists (so it can gate a bulk
83 loop / CI). Non-blocker issues don't stop the gate but populate the review sheet
84 for the human pass. `import-local` independently rejects overlaps/zero-duration, so
85 bad ground truth can't slip into the DB.
86
87 ### How are missed rallies caught? (the worst error)
88 Two complementary checks, because a miss shows up two ways:
89 1. Clean omission (a real hole in the timeline) ? `possible_missed_rally` (gap
90 heuristic).
91 2. Merged/inflated (the missed rally got swallowed into a neighbour's box, leaving
92 no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
93 the second rally inside it.
94 Neither is a guarantee ? they surface regions to review, they don't certify
95 completeness. (A future enhancement: a motion second-opinion over the raw video.)
96
97 ### What's the review sheet?
98 `

```

```

97 (timestamp overlay on) and fill the flagged rows. This is the human gate that turns
98 a delivery into trusted golden labels.
99
100 ### Why two ending fields (`ending_reason` + `last_shot_outcome`)?
101 They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
102 unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
103 last shot went (in / net / out / n_a). A shuttle that flew long is
104 `forced_error/unforced_error` + `out` ? both kept, no information lost.
105
106 ### How do I onboard a brand-new video?
107 1. Get fps into the intake manifest (we provide it to the vendor).
108 2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
109 sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
110 3. The same suite runs for every video, so quality stays consistent at scale.
111
112 ### How do I pull a single rally out to eyeball it?
113 The slicer (indexer):
114 ```bash
115 python -m backend.tools.rally_slicer --video match.mp4 \
116     --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
117 ```
118 Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).
119
120 ### Does coarse frame sampling hurt boundaries?
121 Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122 padding, so we don't flag sampling density.
123
124 ### Will a non-commercial license block export?
125 The video is **kept** in the DB regardless of license, but the default `export` is
126 **commercial-only** ? a non-commercial source (e.g. `CC-BY-NC-SA`) is *excluded*
127 from the eval/training set unless you pass `--include-noncommercial`. So if a video
128 "goes missing" downstream, check its `is_commercial` flag first. (The observability
129 report raises a `license_noncommercial` warning for exactly this case.)
130
131 ### What is the observability report? (*.report.json` / `*.report.md` / `*.summary.md`)
132 Every `validate-delivery`, `convert-cvat`, and `import-local` run now also writes a
133 per-video **observability report** (next to the video for `import-local`; into the
134 `--out-dir` / next to the canonical CSV otherwise):
135 - **`<id>.report.json`** ? machine-readable: every stage, metric, and finding. Diff
136 two runs to see what changed between deliveries.
137 - **`<id>.report.md`** ? the **technical** report: parse / validation / registration
138 stages plus **findings** (one per delivery issue, each citing this FAQ). Read this
139 to debug a delivery.
140 - **`<id>.summary.md`** ? a **delivery summary** for the vendor / data team: the
141 rally count and coverage plus the plain-language list of rallies to review ? no
142 internal codes or severities.
143 This is in addition to the existing `<id>_report.md` / `_report.json` / `_review.csv`
144 (the human review sheet), which are unchanged.
145

```

10. user (2026-06-12T09:03:36.335Z)

```

1 CVAT rally conversion report ? annotations.xml
2   fps used      : 59.940
3   sampling step : 30 frames (~0.50s)
4   rallies parsed : 24
5   issues       : 5 (1 blocker)
6
7 [BLOCKER]
8   - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies
must be disjoint.
9
10 [HIGH]
11   - (coarse_sampling) Frames sampled every 30 (~0.50s at 59.94fps); rally boundaries
carry +/-0.50s uncertainty, exceeding the +/-0.3s spec.

```

```

12
13     [MEDIUM]
14     - (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low ? the span
may be inflated by a stray box.
15     - (shot_density) Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low ? the span
may be inflated by a stray box.
16     - (shot_density) Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low ? the span
may be inflated by a stray box.
17

```

11. assistant (2026-06-12T09:03:39.490Z)

Excellent! Now let me examine the ingestion code itself and look for the parser:

12. user (2026-06-12T09:03:39.871Z)

```

1     import argparse
2     import sys
3     from src.core.config import load_config, get_database_path
4     from src.core.db import SportsDatabase
5     from src.parsers.badminton.coachai import CoachAIBadmintonParser
6
7     def main():
8         parser = argparse.ArgumentParser(description="Ingest external datasets into the
curation staging area")
9         parser.add_argument("--sport", required=True, choices=["badminton"], help="Sport
type")
10        parser.add_argument("--parser", required=True, choices=["coachai"], help="Parser
type to use")
11        parser.add_argument("--file-path", required=True, help="Path to raw dataset file")
12        parser.add_argument(
13            "--annotation-license", "--license", dest="annotation_license", default="MIT",
14            help="License of the ANNOTATIONS/dataset (e.g. ShuttleSet MIT). Recorded for "
15            "provenance only; does NOT make the video commercially usable.",
16        )
17        parser.add_argument(
18            "--video-license", dest="video_license", default="Unknown",
19            help="License of the VIDEO FOOTAGE ? this is what gates commercial use. "
20            "Broadcast/third-party video is 'Unknown' (default, non-commercial) unless
"
21            "you actually hold the footage rights.",
22        )
23        parser.add_argument("--video-url", default="https://youtube.com/watch?v=example",
help="URL to the video")
24        parser.add_argument("--match-name", help="Name of the match")
25        parser.add_argument("--video-id", help="Video unique identifier")
26
27        args = parser.parse_args()
28
29        # Load db path from the central config (falls back to the default).
30        # NOTE: this importer is intentionally badminton/CoachAI-only for now; other
31        # sports are populated via `curate import-local` or vendor golden sets.
32        db = SportsDatabase(get_database_path(load_config()))
33
34        if args.sport == "badminton" and args.parser == "coachai":
35            match_parser = CoachAIBadmintonParser()
36            try:
37                video_meta, rallies = match_parser.parse(
38                    args.file_path,
39                    video_id=args.video_id,
40                    video_url=args.video_url,
41                    annotation_license=args.annotation_license,
42                    video_license=args.video_license,
43                    match_name=args.match_name
44                )

```

```

45
46         # Insert into database (by default parsed entries are marked as staging)
47         db.insert_video(video_meta)
48         count = db.insert_badminton_rallies(rallies)
49
50         print(f"Successfully ingested match '{video_meta.match_name}' into
staging:")
51         print(f" - Video ID: {video_meta.video_id}")
52         print(f" - Total Rallies: {count}")
53         print(f" - Video license: {video_meta.license_type.value}")
54         print(f" - Annotation license: {video_meta.annotation_license.value if
video_meta.annotation_license else 'n/a'}")
55         print(f" - Commercial safe: {video_meta.is_commercial}")
56         if not video_meta.is_commercial:
57             print(" ! NOT commercial-clean: this video is EXCLUDED from `curate
export --commercial-only`.")
58             print("    Pass --video-license <held-rights-license> only if you
actually hold footage rights.")
59             except Exception as e:
60                 print(f"Error parsing file: {e}")
61                 sys.exit(1)
62
63     if __name__ == "__main__":
64         main()
65

```

13. user (2026-06-12T09:03:40.179Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:

```

```

35         return 0.0
36     s = str(time_str).strip()
37     if ":" in s:
38         parts = s.split(":")
39         try:
40             if len(parts) == 3:
41                 return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42             if len(parts) == 2:
43                 return float(parts[0]) * 60 + float(parts[1])
44         except ValueError:
45             return 0.0
46     try:
47         return float(s)
48     except ValueError:
49         return 0.0
50

```

14. user (2026-06-12T09:03:40.311Z)

```

1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class BadmintonRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds")
8         end_time: float = Field(..., description="End of rally in seconds")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10        score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11        shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12        ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13        last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15        @model_validator(mode='after')
16        def validate_timestamps(self) -> 'BadmintonRally':
17            if self.start_time < 0:
18                raise ValueError("start_time must be non-negative")
19            if self.end_time <= self.start_time:
20                raise ValueError("end_time must be strictly greater than start_time")
21            return self
22

```

15. assistant (2026-06-12T09:03:42.689Z)

Now let me look at the `curate.py` file which handles the ``import-local`` command:

16. user (2026-06-12T09:03:43.273Z)

C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py

17. user (2026-06-12T09:03:45.278Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple

```

```

8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.
41            rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43            if not rows:
44                print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                return
46
47            data = [[sport, status, cnt] for sport, status, cnt in rows]
48            print("\n=== Dataset Ingestion Status ===")
49            print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50            print()
51
52        @staticmethod
53        def _annotation_key(sport_str, a):
54            """The per-sport primary-key value of one annotation (for duplicate
detection)."""
55            if sport_str in ("badminton", "pickleball", "tabletennis"):
56                return getattr(a, "rally_number", None)
57            if sport_str == "tennis":
58                return getattr(a, "point_number", None)
59            if sport_str == "cricket":
60                return (getattr(a, "over", None), getattr(a, "ball", None))
61            return None
62
63        def _guard_import(self, sport_str, video_id, annotations, force=False):
64            """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
atomic)."""
65            # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both
66            # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
67            # video row with rolled-back annotations. Catch it before touching the DB.
68            keys = [self._annotation_key(sport_str, a) for a in annotations]

```



```

69         keys = [k for k in keys if k is not None]
70         seen, dupes = set(), set()
71         for k in keys:
72             if k in seen:
73                 dupes.add(k)
74             seen.add(k)
75         if dupes:
76             print(f"Error: duplicate rally/point identifier(s) {sorted(dupes)} in the
annotations "
77                   f"(often a fractional id like 7.5 truncating to 7).")
78             print(" -> re-run with --normalize, or normalize first: "
79                   "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
80             sys.exit(1)
81
82         # (2) Golden-label protection ? refuse to silently clobber an existing video's
83         # annotations (filename-derived video_ids collide easily).
84         if self.db.get_video(video_id):
85             try:
86                 n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
87             except Exception:
88                 n = 0
89             if n > 0 and not force:
90                 print(f"Error: video_id '{video_id}' already has {n} annotated
segment(s); "
91                       f"importing would REPLACE them.")
92                 print(" -> pass --force to overwrite, or use a distinct --video-id to
keep both.")
93                 sys.exit(1)
94             if n > 0:
95                 print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
96
97         def import_local(self, args):
98             """Manually registers a local video and annotation file into the database."""
99             sport_str = args.sport.lower()
100             if sport_str not in self.config.get("supported_sports", ["badminton"]):
101                 print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
102                 sys.exit(1)
103
104             # Validate video path
105             if not os.path.exists(args.video_path):
106                 print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
107
108             # Validate annotation file
109             if not os.path.exists(args.annotations_path):
110                 print(f"Error: Annotations file '{args.annotations_path}' not found.")
111                 sys.exit(1)
112
113             match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
114             video_id = args.video_id or match_name.lower().replace(" ", "_")
115
116             # Parse license
117             try:
118                 license_type = LicenseType(args.license)
119             except ValueError:
120                 print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
121                 license_type = LicenseType.UNKNOWN
122
123             is_commercial = self.validator.check_license_commercial(license_type)
124
125             # Optional: normalize a raw CSV annotator export before parsing (drops unusable
126             # rows, rennumbers fractional/duplicate rally ids) so the landmines below can't

```

```

fire.
127         annotations_path = args.annotations_path
128         if getattr(args, "normalize", False) and
annotations_path.lower().endswith(".csv"):
129             from src.cli.normalize_annotations import normalize_file
130             normalized_path = annotations_path + ".normalized.csv"
131             rep = normalize_file(annotations_path, normalized_path)
132             print("=== normalize-annotations ===")
133             print(rep.summary())
134             annotations_path = normalized_path
135
136         # 1. Parse Annotations
137         annotations = []
138         try:
139             annotations = self._parse_local_annotations(sport_str, video_id,
annotations_path)
140         except Exception as e:
141             print(f"Error parsing annotations: {e}")
142             sys.exit(1)
143
144         # 2. Run Validation
145         is_valid = True
146         errors = []
147         if sport_str == "badminton":
148             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
149         elif sport_str == "tennis":
150             is_valid, errors = self.validator.validate_tennis_points(annotations)
151         elif sport_str == "cricket":
152             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
153         elif sport_str == "pickleball":
154             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
155         elif sport_str == "tabletennis":
156             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
157
158         if not is_valid:
159             print(f"Error: Annotations failed validation!")
160             for err in errors:
161                 print(f"  - {err}")
162             sys.exit(1)
163
164         # 2b. Pre-insert safety ? checked BEFORE any DB write, so a rejection never
165         # leaves a half-updated DB (a committed video row with rolled-back annotations).
166         self._guard_import(sport_str, video_id, annotations, force=getattr(args,
"force", False))
167
168         # 3. Create Video Metadata
169         video_meta = VideoMetadata(
170             video_id=video_id,
171             sport=SportType(sport_str),
172             video_url=None,
173             local_path=args.video_path,
174             license_type=license_type,
175             license_url=None,
176             is_commercial=is_commercial,
177             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
178             match_name=match_name,
179             fps=args.fps,
180             resolution=args.resolution
181         )
182
183         # 4. Insert into Database
184         self.db.insert_video(video_meta)
185
186         count = 0

```

```

187         if sport_str == "badminton":
188             count = self.db.insert_badminton_rallies(annotations)
189         elif sport_str == "tennis":
190             count = self.db.insert_tennis_points(annotations)
191         elif sport_str == "cricket":
192             count = self.db.insert_cricket_deliveries(annotations)
193         elif sport_str == "pickleball":
194             count = self.db.insert_pickleball_rallies(annotations)
195         elif sport_str == "tabletennis":
196             count = self.db.insert_tabletennis_rallies(annotations)
197
198         print(f"\nSuccessfully imported local video '{match_name}':")
199         print(f"   - Video ID: {video_id}")
200         print(f"   - Sport: {sport_str}")

```

18. user (2026-06-12T09:03:47.989Z)

```

200         print(f"   - Sport: {sport_str}")
201         print(f"   - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
202         print(f"   - Registered {count} annotated segments.\n")
203
204         # Per-video observability report (next to the video). Never raises.
205         from src.reporting import emit_import_report
206         src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
207         report_paths = emit_import_report(
208             video_meta_obj=video_meta, annotations=annotations, is_valid=True,
209             errors=[], count=count, sport=sport_str, source_format=src_fmt,
210             fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
211         )
212         if report_paths:
213             print(f"   - Observability report: {report_paths.get('technical_md')}")
214             print(f"   - Customer summary      : {report_paths.get('customer_md')}\n")
215
216         def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
217             """Parses simple JSON or CSV annotations files."""
218             parsed_items = []
219             ext = os.path.splitext(filepath)[1].lower()
220
221             if ext == '.json':
222                 with open(filepath, 'r', encoding='utf-8') as f:
223                     data = json.load(f)
224
225                 if not isinstance(data, list):
226                     raise ValueError("JSON file must contain a list of objects.")
227
228                 # Uses the same safe casts as the CSV path: a missing/blank required
229                 # time raises a clear ValueError (caught by the caller) instead of a
230                 # bare KeyError, and optional ints degrade to sensible defaults.
231                 for idx, obj in enumerate(data):
232                     if sport == "badminton":
233                         parsed_items.append(BadmintonRally(
234                             video_id=video_id,
235                             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
236                             start_time=_safe_float(obj.get("start_time")),
237                             end_time=_safe_float(obj.get("end_time")),
238                             score_before=obj.get("score_before"),
239                             score_after=obj.get("score_after"),
240                             shots_count=_safe_int(obj.get("shots_count"), None),
241                             ending_reason=obj.get("ending_reason"),
242                             last_shot_outcome=obj.get("last_shot_outcome")
243                         ))
244                     elif sport == "tennis":

```

```

245         parsed_items.append(TennisPoint(
246             video_id=video_id,
247             point_number=_safe_int(obj.get("point_number"), idx + 1),
248             start_time=_safe_float(obj.get("start_time")),
249             end_time=_safe_float(obj.get("end_time")),
250             set_score=obj.get("set_score"),
251             game_score=obj.get("game_score"),
252             server=obj.get("server"),
253             ending_type=obj.get("ending_type")
254         ))
255     elif sport == "cricket":
256         parsed_items.append(CricketDelivery(
257             video_id=video_id,
258             over=_safe_int(obj.get("over"), 0),
259             ball=_safe_int(obj.get("ball"), 0),
260             start_time=_safe_float(obj.get("start_time")),
261             end_time=_safe_float(obj.get("end_time")),
262             batsman=obj.get("batsman"),
263             bowler=obj.get("bowler"),
264             runs=_safe_int(obj.get("runs"), None),
265             wicket=bool(obj.get("wicket", False)),
266             extras=_safe_int(obj.get("extras"), 0)
267         ))
268     elif sport == "pickleball":
269         parsed_items.append(PickleballRally(
270             video_id=video_id,
271             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
272             start_time=_safe_float(obj.get("start_time")),
273             end_time=_safe_float(obj.get("end_time")),
274             score_before=obj.get("score_before"),
275             score_after=obj.get("score_after"),
276             shots_count=_safe_int(obj.get("shots_count"), None),
277             ending_reason=obj.get("ending_reason"),
278             last_shot_outcome=obj.get("last_shot_outcome")
279         ))
280     elif sport == "tabletennis":
281         parsed_items.append(TableTennisRally(
282             video_id=video_id,
283             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
284             start_time=_safe_float(obj.get("start_time")),
285             end_time=_safe_float(obj.get("end_time")),
286             score_before=obj.get("score_before"),
287             score_after=obj.get("score_after"),
288             shots_count=_safe_int(obj.get("shots_count"), None),
289             ending_reason=obj.get("ending_reason"),
290             last_shot_outcome=obj.get("last_shot_outcome")
291         ))
292     elif ext == '.csv':
293         with open(filepath, 'r', encoding='utf-8') as f:
294             reader = csv.DictReader(f)
295             # Normalize headers
296             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
297
298         for idx, row in enumerate(reader):
299             if sport == "badminton":
300                 parsed_items.append(BadmintonRally(
301                     video_id=video_id,
302                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
303                     start_time=_safe_float(row.get("start_time")),
304                     end_time=_safe_float(row.get("end_time")),
305                     score_before=row.get("score_before"),
306                     score_after=row.get("score_after"),
307                     shots_count=_safe_int(row.get("shots_count"), None),
308                     ending_reason=row.get("ending_reason"),

```

```

309         last_shot_outcome=row.get("last_shot_outcome")
310     ))
311     elif sport == "tennis":
312         parsed_items.append(TennisPoint(
313             video_id=video_id,
314             point_number=_safe_int(row.get("point_number"), idx + 1),
315             start_time=_safe_float(row.get("start_time")),
316             end_time=_safe_float(row.get("end_time")),
317             set_score=row.get("set_score"),
318             game_score=row.get("game_score"),
319             server=row.get("server"),
320             ending_type=row.get("ending_type")
321         ))
322     elif sport == "cricket":
323         parsed_items.append(CricketDelivery(
324             video_id=video_id,
325             over=_safe_int(row.get("over"), 0),
326             ball=_safe_int(row.get("ball"), 0),
327             start_time=_safe_float(row.get("start_time")),
328             end_time=_safe_float(row.get("end_time")),
329             batsman=row.get("batsman"),
330             bowler=row.get("bowler"),
331             runs=_safe_int(row.get("runs"), None),
332             wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
333             extras=_safe_int(row.get("extras"), 0)
334         ))
335     elif sport == "pickleball":
336         parsed_items.append(PickleballRally(
337             video_id=video_id,
338             rally_number=_safe_int(row.get("rally_number"), idx + 1),
339             start_time=_safe_float(row.get("start_time")),
340             end_time=_safe_float(row.get("end_time")),
341             score_before=row.get("score_before"),
342             score_after=row.get("score_after"),
343             shots_count=_safe_int(row.get("shots_count"), None),
344             ending_reason=row.get("ending_reason"),
345             last_shot_outcome=row.get("last_shot_outcome")
346         ))
347     elif sport == "tabletennis":
348         parsed_items.append(TableTennisRally(
349             video_id=video_id,
350             rally_number=_safe_int(row.get("rally_number"), idx + 1),
351             start_time=_safe_float(row.get("start_time")),
352             end_time=_safe_float(row.get("end_time")),
353             score_before=row.get("score_before"),
354             score_after=row.get("score_after"),
355             shots_count=_safe_int(row.get("shots_count"), None),
356             ending_reason=row.get("ending_reason"),
357             last_shot_outcome=row.get("last_shot_outcome")
358         ))
359     else:
360         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
361
362     return parsed_items
363
364 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
365     """Download one video at best resolution, retrying transient failures.
366
367     A below-min-height result almost always means YouTube rate-limited the
368     request and served a low fallback (not that the video lacks HD), so we
369     retry with a growing backoff. Returns (DownloadResult, None) on success,
370     or (None, failure_reason) once retries are exhausted. A below-min file is
371     discarded each attempt so it never lands in the DB.

```

```

372         """
373         attempts = max(1, args.retries + 1)
374         reason = "unknown error"
375         for i in range(attempts):
376             result = download_best_video(
377                 url, dest_path,
378                 max_height=max_height, max_size_mb=max_size_mb,
379                 cookies_from_browser=args.cookies_from_browser,
380                 cookies_file=args.cookies,
381                 player_client=args.player_client,
382             )
383             if result.success and (result.height is None or result.height >=
args.min_height):
384                 return result, None
385
386             if not result.success:
387                 reason = result.error or "unknown error"
388             else:
389                 reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
390                           f"YouTube served a low format (rate-limited?)")
391             try:
392                 os.remove(result.path)
393             except OSError:
394                 pass
395
396             if i < attempts - 1:
397                 backoff = args.retry_backoff * (i + 1)
398                 print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
399                 time.sleep(backoff)

```

19. assistant (2026-06-12T09:03:50.051Z)

Now let me search for the golden set batch information and look at the database schema to understand how the join works:

20. user (2026-06-12T09:03:51.016Z)

```

C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ANNOTATION_GUIDE.md:## Part 2 ?
The three golden rules
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ROORA_BRIEF.md:**Project:**
racket-sport rally segmentation golden / regression dataset
C:\Users\avidu\Projects\sports-data-collector/docs/INGESTION_PIPELINE.md:
?         golden labels
C:\Users\avidu\Projects\sports-data-collector/docs/INGESTION_PIPELINE.md:a delivery into trusted
golden labels.
C:\Users\avidu\Projects\sports-data-collector/docs/INGESTION_PIPELINE.md:    --labels
golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
C:\Users\avidu\Projects\sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md:#
Positioning: manual (rally-annotator) golden-label ingestion
C:\Users\avidu\Projects\sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md:Position the
collector as the **single canonical system-of-record for golden rally
C:\Users\avidu\Projects\sports-data-
collector/docs/MANUAL_INGESTION_POSITIONING.md:`calibrate_wasb.py::load_golden_gts` reads
`start_time`/`end_time` only).
C:\Users\avidu\Projects\sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md:  hardcoded
`CURATED` (`curate.py`), so the golden-set regression flow restricts the gold
C:\Users\avidu\Projects\sports-data-collector/README.md:- [docs/annotation/](docs/annotation/) ?
the vendor annotation package used to produce golden rally labels:
C:\Users\avidu\Projects\sports-data-collector/src/cli/curate.py:
help="Overwrite an existing video's annotations (default: refuse, to protect golden labels)")
C:\Users\avidu\Projects\sports-data-collector/src/cli/ingest.py:    # sports are populated via
`curate import-local` or vendor golden sets.
C:\Users\avidu\Projects\sports-data-collector/tests/test_import_guard.py:"""import-local must

```

not silently clobber golden labels or crash mid-insert.

C:\Users\avidu\Projects\sports-data-collector/tests/test_import_guard.py: # Original golden labels intact.

21. user (2026-06-12T09:03:53.216Z)

```
1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22    }
23
24
25    class SportsDatabase:
26        def __init__(self, db_path: str):
27            self.db_path = db_path
28            # Ensure target directory exists
29            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30            self._init_db()
31
32        @contextmanager
33        def _get_connection(self):
34            """Yield a SQLite connection and always close it.
35
36            Used as ``with self._get_connection() as conn:``. A plain
37            ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38            transaction but does NOT close the connection, which leaks handles and
39            raises ResourceWarnings. This wrapper guarantees the connection is closed
40            on exit; writers still commit explicitly inside the block.
41            """
42            conn = sqlite3.connect(self.db_path)
43            conn.row_factory = sqlite3.Row
44            # SQLite disables foreign keys per-connection by default; enable so the
45            # ON DELETE CASCADE declarations on the child tables actually fire.
46            conn.execute("PRAGMA foreign_keys = ON")
47            try:
48                yield conn
49            finally:
50                conn.close()
51
52        def _init_db(self):
53            with self._get_connection() as conn:
54                cursor = conn.cursor()
55
56                # 1. Create common videos table
57                cursor.execute("""
58                CREATE TABLE IF NOT EXISTS videos (
59                    video_id TEXT PRIMARY KEY,
```

```

60         sport TEXT NOT NULL,
61         video_url TEXT,
62         local_path TEXT,
63         license_type TEXT NOT NULL,
64         license_url TEXT,
65         annotation_license TEXT,
66         is_commercial INTEGER NOT NULL DEFAULT 0,
67         status TEXT NOT NULL DEFAULT 'staging',
68         match_name TEXT NOT NULL,
69         fps REAL,
70         resolution TEXT
71     )
72     """)
73
74 # 2. Create badminton_rallies table
75 cursor.execute("""
76 CREATE TABLE IF NOT EXISTS badminton_rallies (
77     video_id TEXT NOT NULL,
78     rally_number INTEGER NOT NULL,
79     start_time REAL NOT NULL,
80     end_time REAL NOT NULL,
81     score_before TEXT,
82     score_after TEXT,
83     shots_count INTEGER,
84     ending_reason TEXT,
85     last_shot_outcome TEXT,
86     PRIMARY KEY (video_id, rally_number),
87     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
88 )
89 """)
90
91 # 3. Create tennis_points table
92 cursor.execute("""
93 CREATE TABLE IF NOT EXISTS tennis_points (
94     video_id TEXT NOT NULL,
95     point_number INTEGER NOT NULL,
96     start_time REAL NOT NULL,
97     end_time REAL NOT NULL,
98     set_score TEXT,
99     game_score TEXT,
100    server TEXT,
101    ending_type TEXT,
102    PRIMARY KEY (video_id, point_number),
103    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
104 )
105 """)
106
107 # 4. Create cricket_deliveries table
108 cursor.execute("""
109 CREATE TABLE IF NOT EXISTS cricket_deliveries (
110     video_id TEXT NOT NULL,
111     over_num INTEGER NOT NULL,
112     ball_num INTEGER NOT NULL,
113     start_time REAL NOT NULL,
114     end_time REAL NOT NULL,
115     batsman TEXT,
116     bowler TEXT,
117     runs INTEGER,
118     wicket INTEGER DEFAULT 0,
119     extras INTEGER DEFAULT 0,
120     PRIMARY KEY (video_id, over_num, ball_num),
121     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
122 )
123 """)
124

```



```

125         # 5. Create pickleball_rallies table
126         cursor.execute("""
127         CREATE TABLE IF NOT EXISTS pickleball_rallies (
128             video_id TEXT NOT NULL,
129             rally_number INTEGER NOT NULL,
130             start_time REAL NOT NULL,
131             end_time REAL NOT NULL,
132             score_before TEXT,
133             score_after TEXT,
134             shots_count INTEGER,
135             ending_reason TEXT,
136             last_shot_outcome TEXT,
137             PRIMARY KEY (video_id, rally_number),
138             FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
139         )
140         """)
141
142         # 6. Create tabletennis_rallies table
143         cursor.execute("""
144         CREATE TABLE IF NOT EXISTS tabletennis_rallies (
145             video_id TEXT NOT NULL,
146             rally_number INTEGER NOT NULL,
147             start_time REAL NOT NULL,
148             end_time REAL NOT NULL,
149             score_before TEXT,
150             score_after TEXT,
151             shots_count INTEGER,
152             ending_reason TEXT,
153             last_shot_outcome TEXT,
154             PRIMARY KEY (video_id, rally_number),
155             FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
156         )
157         """)
158
159         # Migration: add last_shot_outcome to rally tables that predate the
160         # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
161         # do it explicitly and idempotently (skip if the column is already there).
162         for tbl in ("badminton_rallies", "pickleball_rallies",
163 "tabletennis_rallies"):
164             existing = [r[1] for r in cursor.execute(f"PRAGMA
165 table_info({tbl}))").fetchall()]
166             if "last_shot_outcome" not in existing:
167                 cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
168 TEXT")
169
170         # Migration: split the conflated single license into video vs annotation
171         # license. annotation_license records the dataset's annotation rights (e.g.
172         # ShuttleSet MIT) for provenance; license_type stays the VIDEO's rights,
173         # which
174         # is what gates commercial use.
175         video_cols = [r[1] for r in cursor.execute("PRAGMA
176 table_info(videos)").fetchall()]
177         if "annotation_license" not in video_cols:
178             cursor.execute("ALTER TABLE videos ADD COLUMN annotation_license TEXT")
179
180         conn.commit()
181
182     def insert_video(self, video: VideoMetadata) -> bool:
183         """Insert a video entry, or update it in place if it already exists.
184
185         Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
186         existing row, which (with foreign keys enabled) would cascade-delete the
187         video's child annotations. UPSERT mutates the row in place and leaves
188         children untouched.
189         """

```

```

185         with self._get_connection() as conn:
186             cursor = conn.cursor()
187             cursor.execute("""
188                 INSERT INTO videos (
189                     video_id, sport, video_url, local_path, license_type,
190                     license_url, annotation_license, is_commercial, status, match_name, fps,
resolution
191                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
192             ON CONFLICT(video_id) DO UPDATE SET
193                 sport=excluded.sport,
194                 video_url=excluded.video_url,
195                 local_path=excluded.local_path,
196                 license_type=excluded.license_type,
197                 license_url=excluded.license_url,
198                 annotation_license=excluded.annotation_license,
199                 is_commercial=excluded.is_commercial,
200                 status=excluded.status,

```

22. assistant (2026-06-12T09:03:57.018Z)

Now let me look for information about video_id assignment and how it's used:

23. user (2026-06-12T09:03:57.746Z)

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ANNOTATION_GUIDE.md:- [] File downloaded as ``<video_id>.csv``, header row present, UTF-8.

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md:One CSV per video (UTF-8), named exactly ``<video_id>.csv`` (we supply the ``video_id``).

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md: re-encode the video ? our pipeline identifies a video by its file checksum.

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md:- Per-video ``fps`` + ``video_id``: provided in the intake manifest ? template

[`INTAKE\_MANIFEST\_TEMPLATE.csv`])(./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the manifest (one row per pilot video) and share its link.**

C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md:- **TODO(avidullu): paste the Drive folder link** where Roora uploads completed ``<video_id>.csv`` files.

C:\Users\avidu\Projects\sports-data-collector\docs\DECISIONS.md:# Generic:

insert_segments(sport, items) / get_segments(sport, video_id) /

C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:python -m src.cli.curate export # ? manifest.json + `<video_id>.csv` (start,end)

C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:

`(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert

C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md: table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on

C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md:The indexer keys videos by **file MD5**, this repo by a human ``video_id``. A

C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md:re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the

24. assistant (2026-06-12T09:03:59.831Z)

Perfect! Let me read that PIPELINE_ROLE document:

25. user (2026-06-12T09:03:59.984Z)

```

1      # Role in the rally-segmenter training pipeline
2
3      This repo is the data producer in a two-repo system. Its job is to curate
4      sports videos with per-rally [start, end) annotations and commercial-license
5      metadata, then hand that data to the badminton-highlight-indexer, which
6      processes, scores, and (Phase 4) trains an offline rally segmenter on it.
7
8      This doc explains why the export and fetch commands exist and how they fit
9      the bigger picture. The full decision records and end-to-end narrative live in the

```

```

10     indexer repo:
11
12     - `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
13     - `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
14       (acquisition policy), ADR-004 (offline learned segmenter).
15
16     ## Why this repo matters
17
18     The indexer's best rally mode refines local detections with a paid Gemini API.
19     The strategic goal is to replace that with an offline model trained on our own
20     commercially-licensed, annotated corpus ? making rally detection free, offline,
21     and self-improving. None of that is possible without a clean, licensable,
22     accurately-labelled, full-resolution dataset. That dataset is what this repo
23     produces.
24
25     ## What the two key commands do (and why)
26
27     - curate export ? emits the indexer's ground-truth format: per-video
28       `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29       Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30       table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31       `is_commercial`, so only licensable data flows
32       downstream. The annotations are temporal-only ? they train/evaluate rally
33       segmentation, not the frame-level shuttle detector (see ADR-002).
34
35     - fetch ? (re)downloads videos at best available resolution, updating
36       the DB in place so annotations and curation status are never lost. Uses
37       authenticated cookies for reliable, unthrottled downloads. The why and the
38       hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40     ## The one contract that matters: video identity
41
42     The indexer keys videos by file MD5, this repo by a human `video_id`. A
43     re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44     exact `local_path`, and any downstream processing must run on that file. This is
45     why acquisition (`fetch`) happens before the indexer processes anything.
46

```

26. assistant (2026-06-12T09:04:02.003Z)

Great! Now let me look for any manifest template or examples:

27. user (2026-06-12T09:04:02.514Z)

```

C:\Users\avidu\Projects\sports-data-collector\data/eval_export/manifest.json
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv

```

28. user (2026-06-12T09:04:04.543Z)

```

1     video_id,sport,format,match_name,source_filename,drive_link,fps,duration_mmss,expected_r
allies,assigned_to,status,notes
2     pilot_badminton_01,badminton,singles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:45,10-12,,todo,deliver as pilot_badminton_01.csv
3     pilot_pickleball_01,pickleball,doubles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:30,10-12,,todo,deliver as pilot_pickleball_01.csv
4     pilot_tabletennis_01,tabletennis,singles,TODO(avidullu): match name,TODO(avidullu):
exact source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:20,10-12,,todo,deliver as pilot_tabletennis_01.csv
5

```

29. assistant (2026-06-12T09:04:07.175Z)

Now let me look for any timestamp normalization or offset handling code:

30. user (2026-06-12T09:04:07.658Z)

```
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        print("  -> re-run
with --normalize, or normalize first: "
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        "python -m
src.cli.normalize_annotations --in <csv> --out clean.csv")
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        # Optional: normalize a
raw CSV annotator export before parsing (drops unusable
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        if getattr(args,
"normalize", False) and annotations_path.lower().endswith(".csv"):
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        from
src.cli.normalize_annotations import normalize_file
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        normalized_path =
annotations_path + ".normalized.csv"
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        rep =
normalize_file(annotations_path, normalized_path)
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        print("===
normalize-annotations ===")
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        annotations_path =
normalized_path
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        # Normalize
headers
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        import_parser.add_argument("
--normalize", action="store_true",
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:
help="Run normalize-annotations on a CSV input first (drops unusable rows, "
C:\Users\avidu\Projects\sports-data-collector\src/cli/curate.py:        # Convert-cvat command ?
normalize a vendor CVAT export into our canonical CSV
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:    """Normalize a
VLC rally-annotator CSV into a clean lean CSV that ``import-local``
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:some point, with
mid-session corrections):
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        python -m
src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:class
NormalizeReport:
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:def
normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]], NormalizeReport]:
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        rep =
NormalizeReport(rows_in=len(rows))
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:def
normalize_file(in_path: str, out_path: str) -> NormalizeReport:
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        out, rep =
normalize_rows(rows)
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        ap =
argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe import-local.")
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        rep =
normalize_file(args.in_path, args.out_path)
C:\Users\avidu\Projects\sports-data-collector\src/cli/normalize_annotations.py:        print("===
normalize-annotations ===")
C:\Users\avidu\Projects\sports-data-collector\src/crawlers/badminton_crawler.py:
# Normalize fieldnames
C:\Users\avidu\Projects\sports-data-collector\src/parsers/badminton/coachai.py:        #
Normalize column names to lowercase/strip
```

31. user (2026-06-12T09:04:09.946Z)

```
1    """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
2    ingests safely.
3
4    Two verified landmines in PARTIAL human annotation data (a video tagged only up to
5    some point, with mid-session corrections):
6
```

```

7      1. An unusable time aborts the WHOLE import (one bad row kills the entire video,
8      via the try/except around _parse_local_annotations in curate.py import-local):
9      a blank/missing time makes ``safe_float_required`` raise; a reversed
10     ``end <= start`` or a negative ``start`` makes the ``BadmintonRally`` pydantic
11     validator raise. -> we DROP such rows, mirroring the downstream indexer loader
12     which already skips ``not end > start``.
13     2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
14     the annotator missed) truncates via ``int(float())`` and COLLIDES on the
15     ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
16     silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
17     original id for traceability.
18
19     It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
20     pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
21     output is always the canonical lean schema::
22
23     rally_number,start_time,end_time,ending_reason,sport[,shots_count]
24
25     plus traceability columns (``original_rally_number``, ``notes``) that import-local
26     ignores. Standalone on purpose: it does NOT touch the DB schema or the
27     rally_number-ordered export. Run it, then feed the output to ``curate import-local``.
28
29     python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
30     """
31     from __future__ import annotations
32
33     import argparse
34     import csv
35     import os
36     from dataclasses import dataclass, field
37     from typing import Any, Dict, List, Optional, Tuple
38
39     # Canonical lean output columns + traceability (import-local reads the first four/
40     # five; original_rally_number + notes are ignored by the importer but kept for humans).
41     OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
42                  "shots_count", "original_rally_number", "notes"]
43
44
45     @dataclass
46     class NormalizeReport:
47         rows_in: int = 0
48         rows_out: int = 0
49         dropped: List[Dict[str, Any]] = field(default_factory=list) # {orig_id, reason,
start, end}
50         renumbered: bool = False
51         fractional_ids: List[str] = field(default_factory=list)
52         duplicate_ids: List[str] = field(default_factory=list)
53         mmss_converted: int = 0
54
55         def summary(self) -> str:
56             lines = [f"rows in: {self.rows_in} -> rows out: {self.rows_out}"]
57             if self.dropped:
58                 lines.append(f"dropped {len(self.dropped)} bad row(s):")
59                 for d in self.dropped:
60                     lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
61                                 f"(start={d['start']!r}, end={d['end']!r})")
62             if self.renumbered:
63                 extra = []
64                 if self.fractional_ids:
65                     extra.append(f"fractional ids {self.fractional_ids}")
66                 if self.duplicate_ids:
67                     extra.append(f"duplicate ids {self.duplicate_ids}")
68                 lines.append("renumbered rallies 1..N by start_time"
69                             + (f" ({'; '.join(extra)})" if extra else ""))
70             if self.mmss_converted:

```

```

71         lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
72         if not self.dropped and not self.renumbered and not self.mmss_converted:
73             lines.append("clean input ? no changes beyond column normalization")
74         return "\n".join(lines)
75
76
77     def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
78         """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
79         if raw is None:
80             return None
81         s = str(raw).strip()
82         if not s:
83             return None
84         try:
85             if ":" in s:
86                 parts = [float(p) for p in s.split(":")]
87                 if len(parts) == 3:
88                     return parts[0] * 3600 + parts[1] * 60 + parts[2], True
89                 if len(parts) == 2:
90                     return parts[0] * 60 + parts[1], True
91                 return None
92             return float(s), False
93         except ValueError:
94             return None
95
96
97     def _primary_reason(raw: Any) -> Tuple[str, str]:
98         """Pipe-delimited 'forced_error|other' -> ('forced_error', 'other'). Blank -> ('',
'')."""
99         if raw is None:
100             return "", ""
101         s = str(raw).strip()
102         if not s:
103             return "", ""
104         parts = [p.strip() for p in s.split("|") if p.strip()]
105         if not parts:
106             return "", ""
107         return parts[0], "; ".join(parts[1:])
108
109
110     def _get(row: Dict[str, Any], *keys: str) -> Any:
111         """First non-empty value among the given (already-lowercased) keys."""
112         for k in keys:
113             v = row.get(k)
114             if v is not None and str(v).strip() != "":
115                 return v
116         return None
117
118
119     def normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]],
NormalizeReport]:
120         """Pure core: lowercase-keyed annotator rows -> clean lean rows + report."""
121         rep = NormalizeReport(rows_in=len(rows))
122         kept: List[Dict[str, Any]] = []
123
124         for row in rows:
125             orig_id = _get(row, "rally_number", "rally")
126             s_parsed = _to_seconds(_get(row, "start_time", "start_mmss", "start"))
127             e_parsed = _to_seconds(_get(row, "end_time", "end_mmss", "end"))
128             start = s_parsed[0] if s_parsed else None
129             end = e_parsed[0] if e_parsed else None
130
131             if start is None:

```

```

132         rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
start_time",
133                             "start": _get(row, "start_time", "start_mmss"), "end":
_get(row, "end_time", "end_mmss")})
134         continue
135     if start < 0:
136         # BadmintonRally's pydantic validator rejects start<0 -> would abort the
137         # whole import; drop it here like any other unusable row.
138         rep.dropped.append({"orig_id": orig_id, "reason": "negative start_time",
139                             "start": start, "end": end})
140         continue
141     if end is None:
142         rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
end_time",
143                             "start": start, "end": _get(row, "end_time",
"end_mmss")})
144         continue
145     if end <= start:
146         rep.dropped.append({"orig_id": orig_id, "reason": "end_time <= start_time",
147                             "start": start, "end": end})
148         continue
149     if (s_parsed and s_parsed[1]):
150         rep.mmss_converted += 1
151     if (e_parsed and e_parsed[1]):
152         rep.mmss_converted += 1
153
154     reason, reason_note = _primary_reason(_get(row, "ending_reason"))
155     note_in = _get(row, "notes", "note", "reviewer_comment")
156     notes = "; ".join([x for x in [reason_note, (str(note_in).strip() if note_in
else "")] if x])
157     kept.append({
158         "orig_id": orig_id, "start": float(start), "end": float(end),
159         "ending_reason": reason, "sport": _get(row, "sport"),
160         "shots_count": _get(row, "shots_count", "shots"), "notes": notes,
161     })
162
163     kept.sort(key=lambda r: r["start"])
164
165     # Renumber 1..N by start order; detect whether ids were fractional / duplicate /
166     # not already a clean 1..N sequence (any of which means import-local would collide).
167     orig_seq = [str(r["orig_id"]) if r["orig_id"] is not None else "" for r in kept]
168     new_seq = [str(i) for i in range(1, len(kept) + 1)]
169     rep.renumbered = orig_seq != new_seq
170     seen: Dict[str, int] = {}
171     for oid in orig_seq:
172         seen[oid] = seen.get(oid, 0) + 1
173     rep.duplicate_ids = sorted({oid for oid, c in seen.items() if c > 1 and oid})
174     rep.fractional_ids = sorted({oid for oid in orig_seq if "." in oid})
175
176     out: List[Dict[str, Any]] = []
177     for i, r in enumerate(kept, start=1):
178         out.append({
179             "rally_number": i,
180             "start_time": f"{r['start']:.3f}",
181             "end_time": f"{r['end']:.3f}",
182             "ending_reason": r["ending_reason"] or "",
183             "sport": r["sport"] or "",
184             "shots_count": "" if r["shots_count"] is None else
str(r["shots_count"]).strip(),
185             "original_rally_number": "" if r["orig_id"] is None else str(r["orig_id"]),
186             "notes": r["notes"],
187         })
188     rep.rows_out = len(out)
189     return out, rep
190

```

```

191
192     def normalize_file(in_path: str, out_path: str) -> NormalizeReport:
193         with open(in_path, "r", newline="", encoding="utf-8-sig") as f:
194             reader = csv.DictReader(f)
195             reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or
[[]])]
196             rows = [{(k or "").lower().strip(): v for k, v in r.items()} for r in reader]
197         out, rep = normalize_rows(rows)
198         os.makedirs(os.path.dirname(os.path.abspath(out_path)) or ".", exist_ok=True)
199         with open(out_path, "w", newline="", encoding="utf-8") as f:
200             w = csv.DictWriter(f, fieldnames=OUT_FIELDS)
201             w.writeheader()
202             w.writerows(out)
203         return rep
204
205
206     def main() -> None:
207         ap = argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe
import-local.")
208         ap.add_argument("--in", dest="in_path", required=True, help="rally-annotator
.rallies.csv")
209         ap.add_argument("--out", dest="out_path", required=True, help="clean lean CSV to
write")
210         args = ap.parse_args()
211         rep = normalize_file(args.in_path, args.out_path)
212         print("=== normalize-annotations ===")
213         print(rep.summary())
214         print(f"wrote -> {args.out_path}")
215
216
217     if __name__ == "__main__":
218         main()
219

```

32. assistant (2026-06-12T09:04:12.138Z)

Now let me check the badminton-highlight-indexer repository for how it receives annotations:

33. user (2026-06-12T09:04:13.122Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer/docs/GOLDEN_SET_IMPLEMENTATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/GOLDEN_SET_PHASE1_VIDEOS.md
C:\Users\avidu\Projects\badminton-highlight-indexer/docs/GOLDEN_SET_VENDORING.md
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_import_smoke.py
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_reingest_safety.py

```

34. user (2026-06-12T09:04:16.643Z)

```

1     # Golden-Set Generation & Vendor Process (v1)
2
3     **Status:** Design v1 . **Date:** 2026-05-30 . **Vendor shortlist added 2026-05-30
($6)**
4     **Owner decisions captured:** licensed/owned video only . rally `[start,end)` depth .
tiered turnaround (fast automated + slow human) . vendor bake-off . **Bangalore/India-local
vendor preference**
5
6     This document defines how we produce **golden-set ground truth** (hand-verified rally
boundaries) for the badminton highlights product, how we **select annotation vendors**, and how
that same machinery powers **on-demand / surprise regression testing** of the production tool.
7
8     > **Companion docs:** the scoring mechanics live in [EVALUATION.md](EVALUATION.md); the
annotated-corpus rationale is [ADR-002](archives/decisions/DECISIONS.md); the decision record
for *this* process is **ADR-006**. Video-capture quality requirements (a dependency of
trustworthy golden sets) are tracked separately in

```


VIDEO_RECORDING_GUIDELINES.md.

9

10 > **Update (2026-06-02):** §6 shortlist done; the founder selected **Roora** (Bengaluru) for the **initial paid pilot** ? a **single-vendor pilot first**, not the full bake-off below. Roora's site/capability/location are verified; **quality is TBD via the pilot**. **Step 0** (the annotation guideline) is authored ? collector repo `docs/annotation/` (`ROORA_BRIEF.md` rally CSV schema + `ANNOTATION_GUIDE.md` illustrated guide + intake-manifest & NDA templates). Pilot scope is **3 sports** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is built. Still pending: the internal **gold key** and the vendor-dependent **GS-3 `HumanVendorProvider`** (GS-1/GS-2/GS-4 already landed ? see `GOLDEN_SET_IMPLEMENTATION.md`)(`GOLDEN_SET_IMPLEMENTATION.md`)).

11

12 ---

13

14 ## 1. The keystone insight: one scorer, three jobs

15

16 A vendor's golden-set output is a list of rally `[start,end)` intervals ? **structurally identical to a prediction** from our pipeline. So the **exact same** deterministic scorer in `backend/eval/metrics.py` (`interval_iou` ? `match_intervals` ? `score`) grades all three of:`

17

18 1. **Product vs. ground truth** ? the precision/recall/F1/IoU we already report (`EVALUATION.md``).

19 2. **Vendor vs. our internal answer key** ? the quality gate during vendor selection.

20 3. **Annotator vs. annotator** ? inter-annotator agreement (IAA), our proxy for "is this even a well-defined task?"

21

22 This symmetry is what makes the whole system small, auditable, and extensible. **No new scoring code is needed** ? only a new **source** of intervals (the vendor).

23

24 ---

25

26 ## 2. Why this also solves the privacy nuance

27

28 The monetization goal (hosted API) carries a "someone views the user's video" concern. This process sidesteps it **by construction**:

29

30 - **Golden sets are built only from footage we license or own** ? starting with the founder's **constantly-growing personal collection** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. **No end-user uploads ever flow to a vendor.**

31 - Regression tests therefore run against a **representative held-out corpus**, not literal user content.

32 - All video egress to any third party passes through a **single provider boundary** (§4), so the security controls (DPA, no-reuse, delete-on-delivery, access logging) live in exactly one place.

33

34 > **Honest tradeoff:** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore **strong evidence**, not a **guarantee**, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see `VIDEO_RECORDING_GUIDELINES.md`, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

35

36 ---

37

38 ## 3. Selecting vendors ? the bake-off

39

40 ### Step 0 ? Write the annotation guideline (highest-leverage artifact)

41 Before contacting anyone, write a precise, example-rich definition of **what a rally is**:

42 - **Start trigger:** e.g. first serve motion vs. shuttle leaving the racket ? pick one, show clips.

43 - **End trigger:** shuttle hits floor/net/out, or umpire call.

44 - **Edge cases:** lets/replays, service faults, between-point walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.

45

46 Rally boundaries are genuinely subjective; **ambiguity here is the #1 cause of bad**

golden sets.** This guideline belongs conceptually with the per-sport `SportProfile`, so it generalizes to tennis/cricket later.

47

48 > **Done (2026-06-02):** authored as the Roora **brief + illustrated guide** in the collector repo `docs/annotation/`. Defines the rally CSV schema and the three golden rules ? ignore all dead time; include **every** rally (incl. service faults & lets); a shot = one racket/paddle/bat contact ? plus an `ending\_reason` taxonomy (winner / forced_error / unforced_error / service_fault / let / other, shared across net-separated sports) and per-sport detail incl. **singles & doubles** for badminton & pickleball.

49

50 **Step 1 ? Build a small internal gold key**

51 One trusted expert (initially: the founder) hand-annotates **5?10** licensed clips** to near-perfect. This is the answer key vendors are graded against ? kept private during the bake-off.

52

53 **Step 2 ? Run the bake-off (parallel pilot)**

54 Send the **same** pilot clips to **2?3** shortlisted vendors** (see §6, favoring Bangalore/India), withholding the key. Grade every submission with **our own harness**:

55

Dimension	How we measure it	Tool
Accuracy	vendor CSV vs. gold key ? P/R/F1, mIoU, boundary error	`eval.score()`
Consistency	2 annotators per vendor ? pairwise IoU (IAA)	`interval_iou`
Cost	quoted ? or \$ per video-minute	vendor quote
Turnaround	actual vs. promised, per tier	observed
Ops friction	file-drop vs. API; compliance with our CSV format	
Security	DPA signed; DPDP/ISO posture; delete-on-delivery	contract review

64

65 **Step 3 ? Decide on measured quality-per-dollar**

66 The bake-off numbers **are** the decision ? not sales decks. Low IAA at a vendor means either a weak vendor **or** a vague guideline (Step 0); investigate before blaming the vendor.

67

68 **Step 4 ? Onboard the winner(s)**

69 Sign DPA + the security controls in §4; codify the guideline as their spec; agree SLAs for both tiers (§5).

70

71 ---

72

73 **4. The extensible architecture ? an `AnnotationProvider` registry**

74

75 This codebase already uses pluggable registries (validators, segmenters, AI providers, sports). The idiomatic move is the same shape for annotation sourcing:

76

```

77 ```python
78 class AnnotationProvider(ABC):
79     def submit(self, video_ref, guideline, sport) -> str:           # returns job_id
80         ...
81     def poll(self, job_id) -> str:                                   #
82         ...
83     def fetch(self, job_id) -> List[Interval]:                     # canonical
84         ...
85     ...
86
87 `fetch()` returns the same `List[Interval]` that `annotations.load_annotations()`
88 already produces, so everything downstream is unchanged.
89
89 Three implementations cover today plus both tiers:
90
91 | Provider | Role | Tier |
92 |---|---|---|
93 | `FileProvider` | Reads a pre-made CSV (formalizes today's manual / ShuttleSet flow) |

```

```

? |
94 | `HumanVendorProvider` | The bake-off winner; authoritative ground truth, corpus
growth, deep audits | **Slow (days)** |
95 | `AutomatedProvider` | An independent strong model labels rallies programmatically |
**Fast (hours)** |
96
97 The tiers are just two providers behind one interface, selectable per call.
98
99 ---
100
101 ## 5. The regression automation (the explicit ask)
102
103 > "given a video, get a golden set created and verify its precision and recall against
the production tool"
104
105 This becomes a thin wrapper over machinery that **already exists**:
106
107 ```python
108 def regress(video, tier="fast", tolerance=0.05):
109     gt = annotation_registry.get(tier).fetch( # NEW: the provider
110         annotation_registry.get(tier).submit(video, GUIDELINE, sport))
111     predictions = process(video) # existing production
pipeline ? DB
112     s = eval.score(predictions, gt) # existing metrics.py,
unchanged
113     regressed = (s.recall < baseline[video].recall - tolerance or
114                 s.precision < baseline[video].precision - tolerance)
115     if regressed: alert(video, s, baseline[video])
116     return s
117 ```
118
119 Only **one new component** (the provider). `run_eval.py`, `harness.py`, and `metrics.py`
are reused verbatim. Snapshot a `baseline.json` per corpus video so "regression" is a measured
delta, not a vibe.
120
121 **Triggers** (all feed the same `regress()`): CI on pipeline changes · on-demand (the
"surprise" case) · scheduled cron · pre-release gate.
122
123 ### ?? The oracle problem (a real correctness trap)
124 The fast-tier `AutomatedProvider` **must be a different model lineage than production**.
If production becomes the offline ActionFormer segmenter (ADR-004) and the oracle is **also**
ActionFormer, they share blind spots ? the test shows green while both are wrong. Pairings that
stay honest:
125
126 - production = offline learned segmenter ? fast-tier oracle = **paid Gemini full-video**
(different lineage).
127 - production = Gemini hybrid ? fast-tier oracle = a **different VLM** or the slow human
tier.
128
129 Treat the fast tier as a **smoke alarm**, not ground truth: any fast-tier "regression"
**escalates to the slow human tier** for confirmation before it blocks a release.
130
131 ---
132
133 ## 6. Vendor shortlist (Bangalore/India-favoring)
134
135 > **Researched 2026-05-30 via a lean manual pass** (targeted web search + direct
site/source verification ? NOT the heavy automated workflow, which failed). **Confidence is
labelled per row.** **No pricing is listed because none could be verified** ? every vendor in
this space quotes per-project, so ?/clip is an offline-conversation item (see §6.3). Treat
single-source rows as **leads to verify on the call*, not vetted recommendations.
136
137 ### 6.1 Tier-1 candidates ? verified India presence + video capability (talk to these
first)
138

```

139 | Vendor | Location (verified) | Video / temporal labeling | Notes for the call |
Confidence |
140 |---|---|---|---|
141 | **iMerit** | **Bengaluru** office (A101 & A118, 43 Residency Rd, 560025; tech hub
opened Oct 2022); also Kolkata HQ + US | Yes ? image/video/text/audio; explicit Sports &
Biomechanics domain on their site | Largest & most enterprise-ready (5,500+ annotators); has a
real **Bangalore** footprint for in-person coordination; ask for a sports/temporal-segmentation
reference + a paid pilot. Higher-touch ? likely higher cost. | **High** |
142 | **TELUS International AI** (ex-**Playment**) | Playment was **Bangalore-founded**
(2015); acquired by TELUS 2021, team retained | Yes ? built on Playment's 2D/3D
image+video+LiDAR CV pipeline | Strong CV/video pedigree, Bangalore roots; but now a large
global BPO ? confirm they'll take a **small startup pilot** and who the India delivery contact
is. | **High** (origin) / **Med** (current India ops shape) |
143 | **Taskmonk** | **Bengaluru** | Yes ? "Video" listed as a labeling modality; platform
+ managed-service workforce | Mid-size, India-local, does both tooling and service ?
potentially the best **cost/agility** fit for a startup pilot. Verify temporal/event labeling
specifically (their heritage is e-commerce catalog tagging). | **Med-High** |
144
145 **### 6.2 Tier-2 ? India video-annotation services, location/specialts to confirm on**
contact
146
147 | Vendor | Stated location | Note | Confidence |
148 |---|---|---|---|
149 | **Infolks** | India (Kerala-based per third parties) | Targets startups/SMBs;
image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate | **Med** |
150 | **Analytics** | India + USA (since 2018) | General data-annotation incl. video; widely
listed | **Med** |
151 | **Objectways** | HQ Scottsdale AZ; **India delivery in Coimbatore** (not Bangalore) |
Does video/LiDAR; India delivery but not Bangalore-local | **Med** |
152 | **Roora** (Roora ML Pvt Ltd) | **Bengaluru** ? HSR Layout (verified, `rooragr.com`) |
Image/video/text annotation; sports industry listed. **Selected for the initial paid pilot**
(2026-06-02) ? existence/capability/location verified; **quality TBD via the pilot**. |
Verified (capability) . pilot pending |
153 | **Tika Data** / **Zuru** | Claimed **Bangalore** (single-source listicle each) |
Smaller/startup annotation shops ? unverified beyond one directory mention; verify they exist &
do temporal video before investing time | **Low** |
154
155 **### 6.3 Explicitly NOT verified / out of scope**
156 - **Pricing for everyone** ? all quote-only; do not assume. Get per-video-minute (or
per-rally) quotes during the offline conversations.
157 - **Karya (Bengaluru)** ? confirmed local and reputable, but **speech/text/translation**
only, no video ? **excluded** for our use case.
158 - **Sports/temporal track record** ? beyond iMerit's stated "Sports & Biomechanics"
domain, vendors' specific **rally-boundary / temporal-event** experience is unconfirmed marketing
copy; probe with the pilot (§3).
159 - **DPDP / DPA / ISO posture** ? not verified per-vendor; make it a checklist item in
the bake-off (§3 Step 2 security row).
160
161 **### 6.4 Recommendation**
162 For the **offline conversations**, start with **iMerit** (Bangalore-local, enterprise
QA, sports domain) and **Taskmonk** (Bangalore-local, likely more startup-friendly cost/agility)
as the two primary contacts, with **TELUS/Playment** as a third if you want a heavyweight CV
pedigree. Run the §3 bake-off across whichever 2/3 agree to a paid pilot, and decide on measured
quality-per-rupee. **Verify every "Confidence: Med/Low" claim on the call before committing.**
163
164 > **Update (2026-06-02):** the founder opted to start with **Roora** (Bangalore-local,
startup-friendly) as a **single-vendor pilot** rather than the iMerit/Taskmonk multi-vendor
bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot
doesn't clear the §3/§4 quality bar.
165
166 > Sources (manual pass, 2026-05-30): iMerit Bengaluru hub ? imerit.net press /
PRNewswire (Oct 2022) + contact listings; Playment/TELUS ? BusinessWire & Inc42 (2021
acquisition, Bangalore-founded); Taskmonk & Karya locations ? company sites + AnalyticsIndiaMag
/ IndiaAI; others ? AnalyticsIndiaMag / Ziloservices / vendor sites. Pricing intentionally
omitted (unverifiable).

```

167
168 ---
169
170 ## 7. Suggested phasing
171
172 | Phase | Deliverable | Depends on |
173 |---|---|---|
174 | 0 | Annotation guideline + internal gold key (5?10 clips) | nothing ? can start now |
175 | 1 | Vendor bake-off ? pick winner(s) | $6 research + guideline |
176 | 2 | `AnnotationProvider` registry + `FileProvider` + `HumanVendorProvider` | winner
onboarded |
177 | 3 | `AutomatedProvider` (independent oracle) + `regress()` + baseline snapshots |
Phase 2 |
178 | 4 | Wire surprise triggers (CI / on-demand / scheduled) | Phase 3 |
179
180 Phase 0 is unblocked today and is the highest-leverage step regardless of vendor choice.
181
182 > **Status (2026-06-02):** Phase 0 guideline ? authored (collector `docs/annotation/`);
internal gold key still pending. Phase 1 is running as a **single-vendor Roora pilot** across
**3 sports** (badminton, pickleball, table tennis) rather than a multi-vendor bake-off. Phase
2's `FileProvider` (GS-1) and Phase 3's `regress()` + automated oracle (GS-2/GS-4) have already
landed; the main remaining slice is the **vendor-dependent `HumanVendorProvider`** (GS-3), with
trigger-wiring (GS-5) after it.
183
184 ---
185
186 ## 8. Open questions
187 1. **Guideline authorship** ? who owns the canonical rally definition, and is one
expert's key enough, or do we want 2-expert consensus for the answer key itself?
188 2. **Corpus representativeness** ? how well does the owned/licensed collection mirror
eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)
189 3. **Baseline drift** ? when production legitimately improves, how/when do we re-
snapshot `baseline.json` so old baselines don't raise false regressions?
190 4. **Fast-tier oracle cost** ? per-video Gemini cost at regression frequency (see
`archives/MONETIZATION_AUDIT.md` $4 for the per-video estimate).
191

```

35. assistant (2026-06-12T09:04:18.561Z)

Now let me check the golden set implementation:

36. user (2026-06-12T09:04:19.133Z)

```

1      # Golden-Set Subproject ? Design & Implementation Overview
2
3      **Status:** In progress . **Date:** 2026-05-30
4      **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6      This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8      > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10     ---
11

```

```

12     ## Architecture in one picture
13
14     ```
15         video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]   ground truth
16                 ?           ?           ?           ?
17                 FileProvider HumanVendor Automated(oracle) ?
18                 (GS-1)      (GS-3)      (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19                                                         ?
20         production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]   predictions
21                                                         ?
22                                                         regress(video, tier) (GS-2)
compares the two
23     ```
24
25     **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27     ---
28
29     ## PR breakdown
30
31     | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32     |---|---|---|---|---|
33     | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34     | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35     | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36     | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37     | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39     Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41     ---
42
43     ## Slice details
44
45     ### GS-1 ? Provider registry + FileProvider ? (this PR)
46     - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate`) convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47     - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48     - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49     - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51     ### GS-2 ? Regression runner (next)
52     `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when

```

```

precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54     ### GS-3 ? HumanVendorProvider (after your vendor pick)
55     A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
56     fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
57     limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
58     ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
59     An independent strong labeler for the fast/CI tier. **Must be a different model lineage
60     than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
61     a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
62     own.
63     ### GS-5 ? Regression triggers
64     Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the
65     "surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so
66     legitimate production improvements can rebaseline instead of raising false regressions.
67     ---
68     ## Conventions
69     - **Branch per slice**, one PR each, tests green before PR, full suite must stay green.
70     - New runtime deps: **none** through GS-2 (GS-3 may add a vendor SDK; GS-4 may reuse the
71     existing `google-genai` for a Gemini oracle).
72     - All ground-truth video stays **licensed/owned** ? see the privacy section of the
73     design doc.
74

```

37. assistant (2026-06-12T09:04:21.761Z)

Perfect! Now let me check for the golden set phases video information:

38. user (2026-06-12T09:04:22.330Z)

```

1     # Phase-1 Golden-Set Video Collection Spec
2
3     **Status:** Design v1 . **Date:** 2026-06-02
4     **Companion docs:** [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (vendor process,
5     ADR-006) . [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) (capture spec the
6     user-facing guide derives from) . scoring in [EVALUATION.md](EVALUATION.md).
7
8     What footage to gather for the **first round** of golden-set creation so the trained
9     rally segmenter reaches good precision/recall ? and so the eval number actually predicts
10    production. Golden-set creation is **iterative**: expect multiple rounds, expanding toward
11    whatever axis round-1 metrics show is weakest.
12    ---
13    ## 1. The principle that dominates everything: match the deployment distribution
14
15    The CV literature is consistent: **models trained on broadcast footage fail to
16    generalize to self-recorded amateur video** ? broadcast uses a "straight, fixed camera with
17    minimum variability," while real non-elite footage has limited coverage, occlusions, and
18    inconsistent viewpoints ([team-sports
19    survey](https://link.springer.com/article/10.1007/s10462-024-10934-9), [unstructured-video event
20    detection](https://arxiv.org/pdf/2002.08097)). This is the **same failure mode as ADR-004**
21    (broadcast cameras *track players*, so player pixel-motion inverts during rallies).
22
23    Our product ingests **users' own match recordings** (a fixed phone/GoPro on a tripod).
24    Therefore:
25
26    - The golden set ? **especially the held-out eval split** ? must be dominated by
27    **static, self-recorded, full-court** footage that mirrors real users.
28
29    - **ShuttleSet / broadcast** footage is fine as *training* augmentation and as a

```

contrast set, but must be **minimal in eval**. Measuring precision/recall on broadcast yields a number that does **not** predict production.

18
19 ---
20
21 ## 2. Primary recording profile (also the basis of the user-facing capture guide)

22
23 Grounded in the badminton-CV setup literature (camera ~2 m behind the baseline, ~4.5 m elevated, ~30° down-tilt, whole court + headroom above the net ?
[source](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)):

24
25 | Setting | Spec | Why |
26 |---|---|---|
27 | Mount | Fixed tripod; **no pan/zoom** during play | Avoids the ADR-004 camera-tracking trap |
28 | Position | Behind one baseline/end, centered | Full-court, symmetric view |
29 | Height | **3?5 m** elevated (~4.5 m ideal) | De-occludes the far player & net |
30 | Tilt | **~30° down** | Optimal coverage, low keystone |
31 | Framing | Entire court **headroom** above the net; players never leave frame |
Detection + court-relevance need it |
32 | Lighting | Even; no backlight/glare | Reduces blur & false motion |
33

34 Most of round 1 should use this profile; a minority deliberately varies it (§4) to probe robustness.

35
36 ---
37
38 ## 3. Resolution & frame rate (per sport)

39
40 A controlled study: table-tennis ball detection was **22.5% @ 30 fps ? 93.2% @ 60 fps** (120 fps ? 60 fps) ([Nature Sci Rep](https://www.nature.com/articles/s41598-023-42966-6)); OpenTTGames records at 120 fps. The ball/shuttle is the camera-invariant rally signal (our WASB substrate, ADR-001), so fps is **not** a nicety for fast sports.

41
42 | Sport | Min resolution | Recommended fps | Note |
43 |---|---|---|---|
44 | Badminton | 1080p (720p floor) | 30 ok, **50/60 better** | Shuttle decelerates fast; 60 sharpens smashes |
45 | Pickleball | 1080p | **30?60** | Ball slower than TT; 30 likely fine, 60 safer |
46 | Table tennis | 1080p | **60 min (120 ideal)** | 30 fps collapses ball detection |
47
48 > These are **literature-backed starting points**; the published minimums get finalized empirically against our golden set (VIDEO_RECORDING_GUIDELINES.md §3).

49
50 ---
51
52 ## 4. The Phase-1 diversity matrix

53
54 Cover the **high-impact axes** now; don't chase rare conditions in round 1. "Guards" = whether the axis mostly protects **P**recision (not calling dead-time a rally) or **R**ecall (not missing rallies).

55
56 | Dimension | Vary across? | Guards | Round-1 target |
57 |---|---|---|---|
58 | **Sport** | badminton, pickleball, table tennis | both | all 3 |
59 | **Format** | singles **and** doubles (badminton, pickleball) | R | both for those 2 |
60 | **Camera profile** | mostly primary-static; 1?2 side-elevated; 1?2 lower/phone-height; **1?2 broadcast (contrast only)** | both | ~80% static / ~20% variants |
61 | **Venue / court color** | wood, green, blue synthetic | P | ?3 venues |
62 | **Lighting** | bright gym, dim club, mixed/backlit | both | ?3 conditions |
63 | **Background clutter** | empty hall vs people moving behind court | **P** (crowd motion ? false positives) | both |
64 | **Players** | skill, attire colors, body types, L/R-handed, skin tone/age | R | spread; don't reuse one pair |
65 | **Rally edge cases** | service faults, lets/replays, 1?2-shot & long rallies, net-


```

cords, coaching pauses, warm-up | boundary **P/R** | ensure present |
66 | **Resolution/fps ablation** | same match re-encoded 480/720/1080p; 30 vs 60 fps |
calibration | 1 matched set/sport |
67
68 ---
69
70 ## 5. Split it so the precision/recall number is honest (no train/eval leakage)
71
72 The #1 video-ML mistake: the **same match in train and eval inflates the score** ([data-
leakage guide](https://towardsdatascience.com/data-science-mistakes-to-avoid-data-
leakage-e447f88aae1c/)).
73
74 - **Split by match / recording session ? never by clip.** All clips from one match go
entirely to train *or* eval.
75 - **Hold out whole *conditions*** (a venue, a camera profile, a player pair the model
never saw) so eval measures generalization, not memorization.
76 - Reserve **~? of each sport as a deployment-like held-out eval set** (static, self-
recorded). Broadcast/ShuttleSet may enter train but should be **minimal in eval**.
77
78 > **Guardrails ? TODO (design with the user):** train and eval live in **separate
catalogs** today, kept apart by convention. We want hard guardrails ? permissions / explicit
approval / re-confirmation at ingest & export ? so a video can **never** silently land in both.
Tracked in BACKLOG.md; to be designed in a later session.
79
80 ---
81
82 ## 6. Concrete "round 1" shopping list (per sport)
83
84 Aligns with the 12?15-videos/sport scale plan; the 3-clip Roora calibration is a subset
of this.
85
86 - **8 static-primary** clips spanning venues / lighting / skill (?4 singles, 4 doubles
where applicable)
87 - **2 camera-variant** clips (side-elevated, lower height) ? robustness probe
88 - **1 resolution/fps ablation set** (one match @ 480/720/1080p, 30 & 60 fps)
89 - **1 broadcast contrast** clip (ShuttleSet) ? *train/contrast only*
90 - Each ~2?4 min, **deliberately including the edge cases** above ? **~12 clips/sport** ;
hold out **4 whole matches** for eval.
91
92 ---
93
94 ## 7. Deliberately defer to later rounds
95
96 Outdoor courts, phone-handheld/jitter, extreme angles, 4K, unusual net/court variants,
crowd-heavy tournaments. Add these once round-1 precision/recall shows the weakest axis, then
expand toward it (find the quality "knee" ? VIDEO_RECORDING_GUIDELINES.md §3).
97
98 ---
99
100 ## 8. Two cautions specific to precision (the stated goal)
101
102 - **Backgrounds with people moving** behind the court are the biggest false-positive
driver. Include some so the model learns to reject them ? and make sure the annotation
guideline's "ignore all dead time" rule is airtight.
103 - **Warm-up / knock-up / practice swings** look like rallies. Capture some, and confirm
the vendor **excludes** them (the Roora guide now has an explicit rule + a how-to-spot-it tip).
Early precision is bounded more by **guideline / inter-annotator consistency (IAA)** than by raw
data volume.
104
105 ---
106
107 ## 9. Sources
108
109 - [Team-sports action-recognition survey ? Springer
2024](https://link.springer.com/article/10.1007/s10462-024-10934-9)

```

110 - [Unstructured-video event detection ? arXiv
2002.08097](https://arxiv.org/pdf/2002.08097)
111 - [Badminton service-height CV setup (camera 4.5 m / 30°) ?
PMC](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)
112 - [VideoBadminton dataset ? arXiv 2403.12385](https://arxiv.org/html/2403.12385v1)
113 - [Table-tennis frame-rate vs detection accuracy ? Nature Sci
Rep](https://www.nature.com/articles/s41598-023-42966-6)
114 - [OpenTTGames (120 fps) dataset](https://lab.osai.ai/)
115 - [Action-spotting / segmentation survey ? arXiv
2505.03991](https://arxiv.org/html/2505.03991v2)
116 - [Data-leakage in train/test splits](https://towardsdatascience.com/data-science-
mistakes-to-avoid-data-leakage-e447f88aae1c/)
117

39. assistant (2026-06-12T09:04:25.062Z)

Now let me search for any convert-cvat functionality and look at the data that Roora might have already sent:

40. user (2026-06-12T09:04:25.914Z)

```
C:\Users\avidu\Projects\sports-data-collector/.git/logs/refs/heads/fix/cvat-empty-corrupt-zip-
gate
C:\Users\avidu\Projects\sports-data-collector/.git/refs/heads/fix/cvat-empty-corrupt-zip-gate
C:\Users\avidu\Projects\sports-data-collector/src/parsers/cvat
C:\Users\avidu\Projects\sports-data-collector/src/parsers/cvat/rally_cvat.py
C:\Users\avidu\Projects\sports-data-
collector/src/parsers/cvat/__pycache__/rally_cvat.cpython-313.pyc
C:\Users\avidu\Projects\sports-data-collector/tests/test_cvat_parser.py
C:\Users\avidu\Projects\sports-data-collector/tests/test_cvat_robustness.py
C:\Users\avidu\Projects\sports-data-
collector/tests/__pycache__/test_cvat_parser.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\sports-data-
collector/tests/__pycache__/test_cvat_robustness.cpython-313-pytest-9.0.3.pyc
```

41. user (2026-06-12T09:04:28.104Z)

```
1      """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3      Why this exists
4      -----
5      Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6      per-rally CSV that imports directly via curate import-local. A vendor may
7      instead do the work in CVAT and hand back the CVAT export: per-frame bounding
8      boxes, with the rally fields (rally_number, shots_count,
9      ending_reason ...) carried as CVAT object attributes on each box. The box
10     geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11     metadata to.
12
13     This module is the ingestion-side adapter that absorbs that vendor format so the
14     modeling repo never has to. It does two things, both deliberately on our side
15     so we fix mechanical problems rather than renegotiating them:
16
17     1. Recompute time from the frame range. A rally's interval is taken from the
18     first/last frame a box for that rally appears on:
19     seconds = frame / fps at the video's true fps. We ignore any
20     start_time/end_time attributes the vendor wrote ? a CVAT job that
21     samples every step frames and assumes 30 fps yields systematically wrong
22     times (we have seen start_time == frame / (step * 30), i.e. off by the
23     sampling step at the wrong fps). The frame index is reliable; the vendor's
24     seconds are not.
25
26     2. Surface ? never silently fix ? content defects. Overlapping rallies,
27     zero-duration rallies, ending_reason values outside the agreed set, and
```

```

28         too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29         rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either **image mode**
32     (<image><box>...</>) or **track/interpolation mode** (<track><box frame=...></>).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import os
39     import re
40     import zipfile
41     import xml.etree.ElementTree as ET
42     from collections import Counter
43     from dataclasses import dataclass, field
44     from typing import Any, Dict, List, Optional, Tuple
45
46     from src.core.parsing import safe_int
47     from src.core.downloader import probe_resolution_fps
48
49     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
50     # Kept here as a soft check only (a warning, never a hard failure) so this
51     # adapter does not depend on the annotation-spec wording.
52     DEFAULT_ALLOWED_REASONS = {
53         "winner",
54         "forced_error",
55         "unforced_error",
56         "service_fault",
57         "let",
58         "other",
59     }
60
61     # The observational outcome of the last shot ? orthogonal to ending_reason
62     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
63     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
64
65     # CVAT attribute names we map onto canonical columns. Vendor label schemas
66     # drift ? suffixed names like "score_before (A-B)", and singular/plural or
67     # case/seperator variants like "shot_count" (the Roora deliveries) for
68     # "shots_count" ? so both these aliases and the incoming attribute names are
69     # folded through `_fold_attr_name` before matching.
70     _ATTR_ALIASES = {
71         "rally_number": ("rally_number",),
72         "shots_count": ("shots_count", "shot_count"),
73         "ending_reason": ("ending_reason",),
74         "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
75         (in/net/out)"),
76         "score_before": ("score_before", "score_before (a-b)"),
77         "score_after": ("score_after", "score_after (a-b)"),
78         "notes": ("notes", "note"),
79     }
80
81     def _fold_attr_name(name: str) -> str:
82         """Fold a CVAT attribute name to a tolerant matching key.
83
84         Case ("Shot_Count"), separators ("shot count", "shot-count"), parenthesized
85         hints ("score_before (A-B)") and singular/plural ("shot_count" vs
86         "shots_count") all fold to the same key, so a vendor's naming drift cannot
87         silently drop a column.
88         """
89         s = re.sub(r"\.(.*?)", " ", (name or "").lower())
90         tokens = [t for t in re.split(r"[\s_-]+", s) if t]
91         return "_".join(

```

```

92         t[:-1] if len(t) > 1 and t.endswith("s") and not t.endswith("ss") else t
93         for t in tokens)
94
95
96     _FOLDED_ALIASES = {canon: tuple(dict.fromkeys(_fold_attr_name(a) for a in aliases))
97                          for canon, aliases in _ATTR_ALIASES.items()}
98
99     CANONICAL_COLUMNS = [
100         "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
101         "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
102         "notes",
103     ]
104
105     @dataclass
106     class RallyRecord:
107         """One rally recovered from the CVAT export, time re-derived from frames."""
108         rally_number: int
109         start_time: float          # seconds, = frame_min / fps
110         end_time: float            # seconds, = frame_max / fps (see padding note)
111         shots_count: Optional[int]
112         ending_reason: Optional[str]
113         last_shot_outcome: Optional[str]
114         score_before: Optional[str]
115         score_after: Optional[str]
116         notes: Optional[str]
117         frame_min: int
118         frame_max: int
119         n_boxes: int
120         vendor_start_time: Optional[str] = None    # the (discarded) attribute value
121         vendor_end_time: Optional[str] = None
122
123         @property
124         def duration(self) -> float:
125             return self.end_time - self.start_time
126
127
128     @dataclass
129     class Issue:
130         """A defect found in the delivery. ``blocker`` issues stop a clean import."""
131         severity: str    # 'blocker' | 'high' | 'medium' | 'low'
132         code: str
133         message: str
134         rally_number: Optional[int] = None
135
136
137     @dataclass
138     class ConversionResult:
139         records: List[RallyRecord]
140         issues: List[Issue]
141         fps: float
142         step: int
143         meta: Dict[str, Any] = field(default_factory=dict)
144
145         @property
146         def n_blockers(self) -> int:
147             return sum(1 for i in self.issues if i.severity == "blocker")
148
149
150     def _mmss(seconds: float) -> str:
151         """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
152         m = int(seconds // 60)
153         s = seconds - m * 60
154         return f"{m}:{s:06.3f}"
155

```

```

156
157 def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
158     """Look up a canonical field in a folded-key attrs dict (see _fold_attr_name)."""
159     for alias in _FOLDED_ALIASES[canonical]:
160         if alias in attrs and attrs[alias] != "":
161             return attrs[alias]
162     return None
163
164
165 def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
166     """Parse a CVAT XML export into (meta, boxes).
167
168     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
169     entry per visible box, from either image-mode or track-mode exports. Attr
170     names are folded via ``_fold_attr_name`` (lowercase, separators unified,
171     parenthesized hints dropped, plural-tolerant), not the raw vendor spelling.
172     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
173     """
174     if not os.path.exists(xml_path):
175         raise FileNotFoundError(f"CVAT export not found: {xml_path}")
176
177     # Accept the standard CVAT delivery artifact (a .zip containing annotations.xml) as
178     # well as a bare .xml. Corrupt/unparseable XML is surfaced as a ValueError (a clean
179     # gate failure) rather than an uncaught ParseError traceback.
180     try:
181         if zipfile.is_zipfile(xml_path):
182             with zipfile.ZipFile(xml_path) as zf:
183                 names = ([n for n in zf.namelist() if n.endswith("annotations.xml")]
184                        or [n for n in zf.namelist() if n.endswith(".xml")])
185                 if not names:
186                     raise ValueError(f"CVAT zip contains no annotations.xml:
{xml_path}")
187                 root = ET.fromstring(zf.read(names[0]))
188             else:
189                 root = ET.parse(xml_path).getroot()
190     except ET.ParseError as e:
191         raise ValueError(f"CVAT XML is corrupt/unparseable ({xml_path}): {e}")
192     meta: Dict[str, Any] = {}
193
194     job = root.find("./job")
195     if job is not None:
196         stop = job.findtext("stop_frame")
197         if stop is not None and stop.strip().isdigit():
198             meta["stop_frame"] = int(stop.strip())
199         ff = job.findtext("frame_filter") or ""
200         if "step=" in ff:
201             try:
202                 meta["step"] = int(ff.split("step=")[1].split()[0])
203             except (ValueError, IndexError):
204                 pass
205
206     def _attrs(el: ET.Element) -> Dict[str, str]:
207         out: Dict[str, str] = {}
208         for a in el.findall("attribute"):
209             out[_fold_attr_name(a.get("name"))] = (a.text or "").strip()
210         return out
211
212     boxes: List[Dict[str, Any]] = []
213
214     # Image mode: <image id=frame> ... <box>...</box>
215     for img in root.findall("image"):
216         try:
217             frame = int(img.get("id"))
218         except (TypeError, ValueError):
219             continue

```

```

220         for box in img.findall("box"):
221             boxes.append({"frame": frame, "attrs": _attrs(box)})
222
223     # Track mode: <track> ... <box frame=N outside=0>...</box>
224     for track in root.findall("track"):
225         for box in track.findall("box"):
226             if box.get("outside") == "1":
227                 continue # a track's terminating keyframe carries no real label
228             try:
229                 frame = int(box.get("frame"))
230             except (TypeError, ValueError):
231                 continue
232             boxes.append({"frame": frame, "attrs": _attrs(box)})
233
234     return meta, boxes
235
236
237 def _infer_step(frames: List[int]) -> int:
238     """Most common positive gap between consecutive distinct sampled frames."""
239     uniq = sorted(set(frames))
240     gaps = [b - a for a, b in zip(uniq, uniq[1:])] if b - a > 0]
241     if not gaps:
242         return 1
243     return Counter(gaps).most_common(1)[0][0]
244
245
246 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
247     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
248     if fps is None or fps <= 0:
249         raise ValueError("fps must be a positive number to convert frames to seconds")
250

```

42. assistant (2026-06-12T09:04:30.389Z)

Now let me check what was actually delivered (looking at the pilot report):

43. user (2026-06-12T09:04:30.845Z)

```

1      # Delivery validation ? pilot_badminton_01
2
3      - rallies: **24**
4      - issues: **4** (blocker 1, high 0, medium 3, low 0)
5      - structurally-clean rallies: **21/24** (87%)
6      - gate: **BLOCKED**
7
8      ## BLOCKER
9      - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s ? rallies
must be disjoint.
10
11     ## MEDIUM
12     - (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low ? the span may
be inflated by a stray box.
13     - (shot_density) Rally 7: 2 shots over 16.0s (0.12/s) is implausibly low ? the span may
be inflated by a stray box.
14     - (shot_density) Rally 16: 2 shots over 17.5s (0.11/s) is implausibly low ? the span may
be inflated by a stray box.
15
16     ## Rallies to review: [2, 7, 16]

```

44. assistant (2026-06-12T09:04:33.888Z)

Excellent! Now let me check the validator to see the exact checks being done:

45. user (2026-06-12T09:04:34.026Z)

```

1      from typing import List, Tuple, Union
2      from src.core.config import load_config
3      from src.core.schemas.base import VideoMetadata, LicenseType
4      from src.core.schemas.badminton import BadmintonRally
5      from src.core.schemas.tennis import TennisPoint
6      from src.core.schemas.cricket import CricketDelivery
7      from src.core.schemas.pickleball import PickleballRally
8      from src.core.schemas.tabletennis import TableTennisRally
9
10     class DatasetValidator:
11         def __init__(self, config_path: str = "config/config.yaml"):
12             self.config_path = config_path
13             self.commercial_licenses = self._load_licenses_whitelist()
14
15         def _load_licenses_whitelist(self) -> List[str]:
16             # load_config returns {} on a missing/unreadable file, so the default list
17             # below is the single fallback (keep in sync with config.yaml).
18             config = load_config(self.config_path)
19             return config.get(
20                 'commercial_licenses',
21                 ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
22                 "Proprietary"],
23             )
24
25         def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
26             """Determines if a license type is safe for commercial use based on the
27             configuration whitelist."""
28             val = license_type.value if isinstance(license_type, LicenseType) else
29             license_type
30             return val in self.commercial_licenses
31
32         def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
33         List[str]]:
34             """
35             Validates badminton rallies:
36             - Sorted chronologically.
37             - Start time < End time (handled by Pydantic, but double checked).
38             - No temporal overlap between sequential rallies.
39             """
40             errors = []
41             if not rallies:
42                 return True, errors
43
44             # Sort by rally number to check progression
45             sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
46
47             for i in range(len(sorted_rallies)):
48                 r = sorted_rallies[i]
49                 if r.start_time >= r.end_time:
50                     errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
51                     end_time ({r.end_time})")
52
53                 if i > 0:
54                     prev = sorted_rallies[i - 1]
55                     if r.start_time < prev.end_time:
56                         errors.append(
57                             f"Rally overlap detected between Rally {prev.rally_number} "
58                             f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
59                             f"(starts at {r.start_time}s)"
60                         )
61
62             return len(errors) == 0, errors
63
64         def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
65         List[str]]:

```

```

60         errors = []
61         if not points:
62             return True, errors
63
64         sorted_pts = sorted(points, key=lambda p: p.point_number)
65         for i in range(len(sorted_pts)):
66             pt = sorted_pts[i]
67             if pt.start_time >= pt.end_time:
68                 errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
69 ({pt.end_time})")
70
71             if i > 0:
72                 prev = sorted_pts[i - 1]
73                 if pt.start_time < prev.end_time:
74                     errors.append(
75                         f"Point overlap detected: Point {prev.point_number} (ends
76 {prev.end_time}s) "
77                         f"and Point {pt.point_number} (starts {pt.start_time}s)"
78                     )
79             return len(errors) == 0, errors
80
81     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
82 Tuple[bool, List[str]]:
83         errors = []
84         if not deliveries:
85             return True, errors
86
87         # Sort by over, then ball within over
88         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
89         for i in range(len(sorted_delivs)):
90             d = sorted_delivs[i]
91             if d.start_time >= d.end_time:
92                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
93 end ({d.end_time})")
94
95             if i > 0:
96                 prev = sorted_delivs[i - 1]
97                 if d.start_time < prev.end_time:
98                     errors.append(
99                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
100 {prev.end_time}s) "
101                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
102                     )
103             return len(errors) == 0, errors
104
105     def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
106 List[str]]:
107         """Validates pickleball rallies: positive durations and no temporal overlap
108 between chronologically-ordered rallies (same contract as badminton)."""
109         errors = []
110         if not rallies:
111             return True, errors
112
113         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
114         for i in range(len(sorted_rallies)):
115             r = sorted_rallies[i]
116             if r.start_time >= r.end_time:
117                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
118 end_time ({r.end_time})")
119
120             if i > 0:
121                 prev = sorted_rallies[i - 1]
122                 if r.start_time < prev.end_time:
123                     errors.append(
124                         f"Rally overlap detected between Rally {prev.rally_number} "

```



```

118             f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119             f"(starts at {r.start_time}s)"
120         )
121     return len(errors) == 0, errors
122
123     def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
124     Tuple[bool, List[str]]:
125         """Validates table-tennis rallies: positive durations and no temporal overlap
126         between chronologically-ordered rallies (same contract as badminton)."""
127         errors = []
128         if not rallies:
129             return True, errors
130
131         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
132         for i in range(len(sorted_rallies)):
133             r = sorted_rallies[i]
134             if r.start_time >= r.end_time:
135                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
136                 end_time ({r.end_time})")
137             if i > 0:
138                 prev = sorted_rallies[i - 1]
139                 if r.start_time < prev.end_time:
140                     errors.append(
141                         f"Rally overlap detected between Rally {prev.rally_number} "
142                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
143                         f"(starts at {r.start_time}s)"
144                     )
145         return len(errors) == 0, errors

```

46. assistant (2026-06-12T09:04:39.101Z)

Perfect! Now let me look for the CVAT validation checks to see the detailed validation rules:

47. user (2026-06-12T09:04:39.938Z)

```

304:         "blocker", "zero_duration",
329:         "medium", "shot_density",
346:         "blocker", "overlap",
347:         f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
354:         # rally MERGED into its neighbour's span (shot_density/inflated catches
358:         "medium", "possible_missed_rally",

```

48. user (2026-06-12T09:04:42.385Z)

```

250
251     by_rally: Dict[int, List[Dict[str, Any]]] = {}
252     for b in boxes:
253         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
254         if rn is None:
255             continue
256         by_rally.setdefault(rn, []).append(b)
257
258     records: List[RallyRecord] = []
259     for rn in sorted(by_rally):
260         items = by_rally[rn]
261         frames = sorted(b["frame"] for b in items)
262         attrs = items[0]["attrs"]
263         fmin, fmax = frames[0], frames[-1]
264         records.append(RallyRecord(
265             rally_number=rn,
266             start_time=fmin / fps,
267             end_time=fmax / fps,

```

```

268         shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
269         ending_reason=_attr_lookup(attrs, "ending_reason"),
270         last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
271         score_before=_attr_lookup(attrs, "score_before"),
272         score_after=_attr_lookup(attrs, "score_after"),
273         notes=_attr_lookup(attrs, "notes"),
274         frame_min=fmin,
275         frame_max=fmax,
276         n_boxes=len(frames),
277         vendor_start_time=attrs.get("start_time"),
278         vendor_end_time=attrs.get("end_time"),
279     ))
280     return records
281
282
283     def detect_issues(records: List[RallyRecord],
284                       stop_frame: Optional[int] = None,
285                       allowed_reasons: Optional[set] = None,
286                       allowed_outcomes: Optional[set] = None,
287                       max_gap_s: float = 15.0) -> List[Issue]:
288         """Find the defects that block or degrade a clean ground-truth import.
289
290         Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
291         slack of ~0.5s is acceptable for this pipeline because the downstream
292         highlight compiler pads every clip on output (badminton-highlight-indexer,
293         backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
294         absorbs the sampling uncertainty when highlights are trimmed.
295         """
296         allowed = allowed_reasons if allowed_reasons is not None else
297         DEFAULT_ALLOWED_REASONS
298         allowed_out = allowed_outcomes if allowed_outcomes is not None else
299         DEFAULT_ALLOWED_OUTCOMES
300         issues: List[Issue] = []
301
302         ordered = sorted(records, key=lambda r: r.rally_number)
303         for i, r in enumerate(ordered):
304             if r.end_time <= r.start_time:
305                 issues.append(Issue(
306                     "blocker", "zero_duration",
307                     f"Rally {r.rally_number}: end <= start "
308                     f"({frames {r.frame_min}-{r.frame_max}}); single-sample or stray box.",
309                     r.rally_number))
310
311             if r.ending_reason is not None and r.ending_reason not in allowed:
312                 issues.append(Issue(
313                     "medium", "ending_reason_vocab",
314                     f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
315                     f"not in the agreed set {sorted(allowed)}.",
316                     r.rally_number))
317
318             if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
319                 issues.append(Issue(
320                     "low", "outcome_vocab",
321                     f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
322                     f"not in {sorted(allowed_out)}.",
323                     r.rally_number))
324
325             # Inflated span: a long rally with implausibly few shots usually means a
326             # stray/duplicated box stretched the frame range past the real ending.
327             if r.shots_count and r.duration >= 5.0:
328                 sps = r.shots_count / r.duration
329                 if sps < 0.3:
330                     issues.append(Issue(
331                         "medium", "shot_density",
332                         f"Rally {r.rally_number}: {r.shots_count} shots over "

```

```

331         f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
332         f"may be inflated by a stray box.",
333         r.rally_number))
334
335     if stop_frame is not None and r.frame_max > stop_frame:
336         issues.append(Issue(
337             "high", "frame_out_of_range",
338             f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
339             f"last frame {stop_frame}.",
340             r.rally_number))
341
342     if i > 0:
343         prev = ordered[i - 1]
344         if r.start_time < prev.end_time:
345             issues.append(Issue(
346                 "blocker", "overlap",
347                 f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
348                 f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
349                 f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
350                 r.rally_number))
351             elif r.start_time - prev.end_time > max_gap_s:
352                 # A long unlabeled stretch between two rallies is a cheap, label-only
353                 # second opinion for the worst error (a missed rally). It won't catch a
354                 # rally MERGED into its neighbour's span (shot_density/inflated catches
355                 # that), but it does catch an omitted rally that left a visible hole.
356                 gap = r.start_time - prev.end_time
357                 issues.append(Issue(
358                     "medium", "possible_missed_rally",
359                     f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
360                     f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
361                     f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
362                     gap.",
363                     r.rally_number))
364
365     return issues
366
367 def convert(xml_path: str,
368            fps: Optional[float] = None,
369            video_path: Optional[str] = None,
370            step: Optional[int] = None,
371            allowed_reasons: Optional[set] = None) -> ConversionResult:
372     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
373
374     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
375     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
376     that silent guess is exactly the bug this adapter exists to undo.
377     """
378     meta, boxes = parse_cvat_boxes(xml_path)
379
380     if fps is None and video_path:
381         _, _, probed = probe_resolution_fps(video_path)
382         fps = probed
383     if fps is None or fps <= 0:
384         raise ValueError(
385             "Could not determine fps. Pass fps=... or a readable video_path so "
386             "frames can be converted to seconds (seconds = frame / fps).")
387
388     if step is None:
389         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
390
391     records = build_records(boxes, fps)
392     issues = detect_issues(records,
393                           stop_frame=meta.get("stop_frame"),
394                           allowed_reasons=allowed_reasons)
395     return ConversionResult(records=records, issues=issues, fps=fps, step=step,

```

```

meta=meta)
395
396
397     def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
398         """Write records as the canonical per-rally CSV that ``import-local`` reads."""
399         import csv

```

49. assistant (2026-06-12T09:04:44.717Z)

Excellent! Now I have enough information. Let me compile the final comprehensive report:

50. user (2026-06-12T09:05:41.706Z)

Structured output provided successfully

51. assistant (2026-06-12T09:05:50.927Z)

Perfect! I've completed a comprehensive end-to-end mapping of the Roora vendor handoff flow. Here's a summary of what I found:

Key Findings

1. **Annotation Guideline & Vendor Tooling**

- **Spec location:** `sports-data-collector/docs/annotation/ROORA\_BRIEF.md` + `ANNOTATION\_GUIDE.md`
- **Roora contact:** `info@rooragrp.com` (Bengaluru)
- **Recommended tools:** Frame-steppable players (VLC) or temporal-labeling tools (Label Studio, CVAT)
- **Critical constraint:** "Do not trim, clip, or re-encode the video ? our pipeline identifies a video by its file checksum" (ROORA_BRIEF.md §1)

2. **Join Key for Annotations?Video**

- **Primary key:** `(video\_id, rally\_number)`
- **video_id** is human-assigned (from intake manifest)
- **rally_number** is 1-based integer, chronological, no gaps
- Database schema: badminton_rallies table with this as PRIMARY KEY + foreign key to videos table

3. **Timestamp Units & Precision**

- **Format:** Decimal **seconds** (floats)
- **Precision:** ± 0.3 s tolerance (ROORA_BRIEF.md §4)
- **Conversion:** mm:ss.mmm ? seconds via Google Sheet formula OR direct decimal entry
- **Critical code:** `safe\_float\_required()` in `src/core/parsing.py` ? raises if missing (required field)

4. **Golden-Set Batch Status (GS-1...GS-4)**

- **GS-1 Pilot:** 1 badminton video sent to Roora, 24 rallies returned in CVAT XML
- **Quality issues (blocker):** Rally 1 overlaps Rally 2 by 18.52 s ? invalid chronology
- **Root cause:** Coarse CVAT sampling (30-frame step at 59.94 fps ? 0.5 s per frame) caused inflated spans
- **Status:** BLOCKED, awaiting Roora rework

5. **Timestamp Offset/Normalization**

- **No configurable offset correction** currently in the code
- If vendor receives re-encoded video with leading padding, returned timestamps will drift
- **Safeguard:** The spec explicitly forbids re-encoding to prevent this
- Normalization tool available to clean up fractional/duplicate rally_numbers, convert mm:ss formats, drop invalid rows

The three repos have a clear contract: collector handles vendor ingestion + validation, indexer handles scoring/training downstream via the annotation CSV export.